SELF-OPTIMIZING AGENT COORDINATION & RESPONSE SYSTEM

Rajapaksha D.N

IT22311436

BSc (Hons) Degree in Information Technology

Specializing in Software Engineering

Department of Software Engineering, Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

SELF-OPTIMIZING AGENT COORDINATION & RESPONSE SYSTEM

Rajapaksha D.N

IT22311436

Dissertation Submitted In Partial Fulfillment of the Requirements for the Bachelor of Science (Hons) Degree in Information Technology Specializing In Software Engineering

Department of Software Engineering, Faculty of Computing

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

DECLARATION

I declare that this is my own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, I hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute my dissertation, in whole or in part in print, electronic or other medium. I retain the right to use this content in whole or part in future works (such as articles or books).

Name	**Student ID**	**Signature**
D N RAJAPAKSHA	It22311436	

The above candidate has carried out research for the bachelor's degree dissertation under my supervision.

Mr. Vishan Jayasinghearachchi	Supervisor	
Dr. Dharshana Kasthurirathna	Co-Supervisor	

Date: …….

ABSTRACT

Legacy enterprise systems, implemented in technologies such as COBOL and Visual Basic, continue to underpin mission-critical operations across finance, healthcare, and logistics. While operationally stable, these systems are characterised by rigid proprietary interfaces, high maintenance costs, and poor adaptability to modern workflows. Traditional modernisation strategies re-hosting, re-platforming, and full rewrites remain disruptive, expensive, and operationally risky, particularly when source code is unavailable or undocumented.

This dissertation presents the Self-Optimizing Agent Coordination and Response System (SOACRS), a production-grade TypeScript and Node.js microservice forming the central intelligence layer of an Agentic AI Middleware platform. SOACRS receives structured semantic task descriptions, ranks available tools using a multi-factor weighted scoring engine, and returns executable Task Routing Plans to a downstream orchestrator without altering the underlying legacy codebase. The scoring engine integrates four criteria, capability fit, historical SLA likelihood, past reward signals, and normalised cost, producing fully explainable routing decisions for each request.

The system implements a closed-loop continuous learning mechanism. After each execution, the orchestrator reports telemetry back to SOACRS, which updates per-tool performance metrics using an Exponential Weighted Moving Average, enabling routing decisions to improve measurably over time. The architecture follows strict hexagonal layering pure domain models, framework-agnostic application services, Prisma-backed PostgreSQL infrastructure adapters, and thin Express HTTP routes ensuring testability, extensibility, and clear separation of concerns.

Experimental validation demonstrates that the scoring engine correctly prioritises tools with superior SLA histories, that cold-start defaults maintain fair baselines for newly registered tools, and that the telemetry feedback loop produces measurable convergence in delegation accuracy. Plan generation completes within sub-100 millisecond application-layer latency, well within the 500 millisecond SLA target. The proposed system addresses an identified research gap, no existing

framework combines multi-agent coordination, symbolic planning, statistical learning, and non-intrusive RPA-mediated legacy access into a single production-ready microservice with explainability and auditability guarantees.

Keywords: Agentic AI, Multi-Agent Systems, Legacy System Modernisation, Contextual Bandits, Task Routing Plan.

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to my supervisor, Mr. Vishan Jayasinghearachchi, for his unwavering guidance, constructive feedback, and encouragement throughout this research. His detailed reviews of architecture decisions and code quality consistently raised the standard of the deliverables and shaped my approach to systems design.

I am equally grateful to my co-supervisor, Dr. Dharshana Kasthurirathna, whose insightful comments on adaptive learning architectures and multi-agent coordination substantially shaped the theoretical underpinnings of this dissertation. His questions during progress reviews repeatedly pushed me to clarify the mathematical foundations of the scoring engine and to consider edge cases that I would otherwise have overlooked.

I would like to thank my project teammates S. M. A. C. M. Rashad, M. Z. M. Ayyash, and M. A. M. Mufthi whose parallel contributions to the RPA connector, middleware orchestrator, and natural language translation layers created the integrated system context within which SOACRS operates. Collaborative design discussions with this team were instrumental in shaping the interface contracts that SOACRS implements, and their willingness to engage with detailed technical questions across module boundaries made the integration phase substantially less painful than it could have been.

I also acknowledge the Department of Software Engineering at the Sri Lanka Institute of Information Technology for providing the academic environment, infrastructure, and research resources that made this work possible.

Finally, my heartfelt appreciation goes to my family for their patience, understanding, and constant moral support during the intensive periods of this research. Their willingness to accommodate long hours of focused work, particularly during the implementation and testing phases, made the completion of this dissertation possible.

TABLE OF CONTENTS

DECLARATION ..... **i**

ABSTRACT ..... **ii**

ACKNOWLEDGEMENT ..... **iv**

LIST OF FIGURES ..... **ix**

LIST OF TABLES ..... **ix**

LIST OF EQUATIONS ..... **x**

LIST OF LISTINGS ..... **x**

LIST OF ABBREVIATIONS ..... **xi**

1. INTRODUCTION **1**

1.1 Background 1

1.1.1 Traditional modernisation strategies and their limitations 2

1.1.2 Non-intrusive modernisation through rpa 3

1.1.3 The rise of agentic AI 4

1.1.4 The need for self-optimising coordination 6

1.2 Literature Survey 7

1.2.1 Foundational multi-agent coordination protocols 8

1.2.2 Cognitive architectures for planning and reasoning 9

1.2.3 Learning-based coordination and adaptive decision-making 10

1.3 Research Gap 12

1.4 Research Problem 15

1.5 Objectives 16

1.5.1 Main objective 16

1.5.2 Specific objectives 16

1.5.3 Expected contributions 17

2. METHODOLOGY 19

2.1 Overall System Architecture 19

2.1.1 Component interaction and data flow 22

2.1.2 Design principles and technology stack 22

2.2 SOACRS Internal Architecture 23

2.3 Domain Layer 25

2.4 Application Layer 26

2.4.1 Plan builder service 27

2.4.2 Scoring engine 28

2.4.3 Metrics Service and EWMA 29

2.5 Infrastructure Layer 30

2.6 HTTP Layer 32

2.7 Integration with Other LAMA Components 33

2.8 Functional and Non-Functional Requirements 34

2.9 Technology Stack 35

2.10 Commercialization Aspects of the Product 36

2.10.1 Market opportunity and business value 36

2.10.2 Target industries and use cases 36

2.10.3 Commercialization roadmap 37

2.10.4 Competitive advantage 37

2.11 Testing and Evaluation 38

2.11.1 Unit testing - ScoringEngine (SOA-S04) 39

2.11.2 Application testing - PlanBuilder and TelemetryHandler 40

2.11.3 Scenario testing - learning loop (soa-s15) 41

2.11.4 HTTP endpoint testing 42

2.11.5 End-to-end LAMA integration 42
2.11.6 Evaluation metrics 43
2.12 Implementation 44
2.12.1 SOACRS Project Structure 44
2.12.2 Implementing the hexagonal layers 45
2.12.3 Database schema and prisma 46
2.12.4 Integration with other LAMA components 47
3. RESULTS AND DISCUSSION 49
3.1 Quantitative Results 49
3.1.1 SOACRS performance benchmarks 49
3.1.2 LAMA system-level results 50
3.2 Scoring Engine Validation 50
3.3 Learning Loop Validation 51
3.3.1 EWMA convergence 51
3.3.2 Selection switch (SOA-S15) 52
3.3.3 Emergent behaviour during LAMA evaluation 53
3.4 Research Findings 53
3.5 Discussion 54
3.5.1 Architectural strengths 54
3.5.2 Trade-offs and limitations 54
3.5.3 Threats to validity 55
3.6 Summary of Each Student's Contribution 55
4. CONCLUSION 58
4.1 Summary of Contributions 58
4.2 Future Work 59
REFERENCES 61

APPENDICES 63

APPENDIX A: SOACRS Repository Structure 63

APPENDIX B: Sample TaskRoutingPlan JSON 65

APPENDIX C: Prisma Schema (excerpt) 67

APPENDIX D: Sample HTTP Requests 69

LIST OF FIGURES

Figure 1.1: Overall Agentic AI Middleware Architecture 5
Figure 1.2: SOACRS Position within the Middleware Pipeline 7
Figure 2.1: High-Level System Architecture of the LAMA Framework..... 19
Figure 2.2: Execution Flowchart..... 20
Figure 2.3: Hexagonal Architecture of SOACRS 24
Figure 2.4: SOACRS Five-Stage Plan Construction Pipeline 25
Figure 2.5: Telemetry Feedback Loop and EWMA Update 30
Figure 2.6: SOACRS Database Entity Relationship Diagram 31
Figure 2.7: Sequence Diagram - Plan Generation (SOA-S16) 43
Figure 3.1: Test SOA-S04 - Tool Ranking by SLA Likelihood 51
Figure 3.2: Test SOA-S15 - EWMA Convergence over 50 Feedback Events (alpha = 0.20) 52
Figure 3.3: Test SOA-S15 - Selection Switch after Telemetry..... 52

LIST OF TABLES

Table 1.1: Comparative Analysis of Existing Literature versus Proposed System..... 13
Table 2.1: LAMA Technology Stack by Component 23
Table 2.2: SOACRS Domain Layer Type Catalogue 25
Table 2.3: SOACRS Application Layer Service Catalogue..... 27
Table 2.4: Scoring Factor Weights and Cold-Start Defaults..... 29
Table 2.5: SOACRS Database Schema Summary 31
Table 2.6: SOACRS Functional Requirements with Test Coverage 34
Table 2.7: SOACRS Non-Functional Requirements 35
Table 2.8: SOACRS Technology Stack..... 35
Table 2.9: SOACRS Test Suite Summary 38
Table 2.10: ScoringEngine Test Cases (SOA-S04) 40
Table 2.11: PlanBuilderService and TelemetryHandler Test Results..... 41
Table 2.12: HTTP Endpoint and Error Envelope Test Results 42
Table 3.1: SOACRS Performance Benchmarks..... 49
Table 3.2: LAMA System-Level Evaluation Results 50
Table 3.3: Summary of Each Student's Contribution..... 56

LIST OF EQUATIONS

Equation 2.1: SOACRS composite scoring formula 28
Equation 2.2: EWMA reward update 29

LIST OF LISTINGS

Listing 2.1: SemanticTask domain type 26
Listing 2.2: PlanBuilderService.buildPlan() core flow 28
Listing 2.3: planRoutes.ts excerpt 33
Listing 2.4: ScoringEngine SLA preference test (SOA-S04) 40
Listing 2.5: SOACRS project tree (subset) 45
Listing 2.6: SOACRS composition root 46

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
BDI	Belief-Desire-Intention
CNP	Contract Net Protocol
DTO	Data Transfer Object
DXA	Device-Independent Units
ELK	Elasticsearch, Logstash, Kibana
EWMA	Exponential Weighted Moving Average
GUI	Graphical User Interface
HIS	Hospital Information System
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
LLM	Large Language Model
MARL	Multi-Agent Reinforcement Learning
MCP	Model Context Protocol
ORM	Object-Relational Mapper
RPA	Robotic Process Automation
SLA	Service Level Agreement
SOACRS	Self-Optimizing Agent Coordination and Response System
TRP	Task Routing Plan
UCB	Upper Confidence Bound
UUID	Universally Unique Identifier

1. INTRODUCTION

1.1 Background

Legacy enterprise systems remain the operational backbone of mission-critical workflows across finance, healthcare, logistics, and public administration. The persistence of these systems many of which were architected and first deployed in the 1980s and 1990s, stems from a combination of factors: proven operational stability under decades of continuous use, deeply embedded business logic that encodes regulatory and organisational practice in ways that are difficult to extract or replicate, and the prohibitive cost and risk associated with wholesale system replacement. As Bennett [2] observed in his foundational survey of legacy systems, organisations frequently find themselves locked into systems that they cannot easily replace, even as the technology environment around them evolves rapidly.

Paradoxically, the same characteristics that make these systems indispensable are also the source of significant modernisation challenges. Legacy platforms are commonly implemented in obsolete programming languages such as COBOL, RPG, PL/1, or older dialects of Visual Basic, and rely on development methodologies and tooling that are unfamiliar to contemporary software engineers. They typically expose only rigid proprietary interfaces or terminal-based green-screen user interfaces, with no provision for programmatic integration. Documentation is often incomplete, outdated, or absent altogether. The pool of personnel capable of maintaining these systems shrinks every year through retirement and natural attrition, creating an increasing operational risk that organisations are forced to manage rather than eliminate.

In the financial sector, an estimated 43 percent of US banking systems still run COBOL code at their core, processing trillions of dollars in transactions daily. In healthcare, hospital information systems frequently combine modern web frontends with backend cores that have been in continuous operation for two or three decades, accumulating customisations and patches that no single engineer fully understands. In logistics and supply chain operations, customs declaration systems, freight tracking

platforms, and warehouse management software often present a similar profile: stable, mission-critical, and effectively immune to traditional modernisation approaches because the cost of failure is unacceptably high. Government services, particularly in tax administration, social welfare disbursement, and identity management, exhibit the same pattern at even larger scales.

1.1.1 Traditional modernisation strategies and their limitations

The principal modernisation strategies that organisations have adopted over the past three decades fall into three broad categories: re-hosting, re-platforming, and full rewriting. Each approach carries distinct trade-offs in terms of cost, risk, time-to-value, and architectural flexibility.

Re-hosting (also known as 'lift-and-shift') relocates legacy applications onto newer hardware or cloud infrastructure with minimal functional change. This approach offers the lowest initial cost and risk but provides only marginal flexibility gains. The application logic itself remains unchanged, integration interfaces remain proprietary, and the underlying technical debt is preserved rather than addressed. Re-hosting can extend the operational life of a legacy system by five to ten years but does not fundamentally enable modern integration patterns or improve adaptability.

Re-platforming adapts applications to new runtime environments, for example, migrating COBOL code from a mainframe to a Linux-based COBOL runtime, or wrapping legacy modules with REST adapters. This approach typically requires partial rewrites and carries moderate risk of introducing regressions. While it provides better integration potential than re-hosting, it does not fundamentally restructure the application architecture, and the underlying business logic remains coupled to the original technology choices.

Full rewriting grants maximum architectural freedom: the system is re-implemented from scratch using current technologies, frameworks, and design patterns. While this approach can yield substantial long-term benefits, it carries the highest cost and risk profile of any modernisation strategy. It depends critically on the availability of complete and accurate source code or business logic specifications,

which are frequently restricted by intellectual property policies, lost to organisational memory, or never properly documented. Full rewrites have a high probability of introducing defects and typically require extended periods of parallel operation during which both the legacy system and the new replacement run simultaneously, multiplying operational complexity and cost. Industry studies consistently show that full rewrites of large legacy systems exceed budget by 50 to 200 percent and frequently fail to deliver functional parity with the original system within the planned timeframe.

1.1.2 Non-intrusive modernisation through rpa

In response to the limitations of traditional approaches, the past decade has seen the rise of non-intrusive modernisation: wrapping legacy systems with new integration layers without altering the underlying source code. The key insight behind non-intrusive modernisation is that the legacy system itself does not need to change what needs to change is the way modern systems interact with it. By placing automation, translation, and orchestration layers between the legacy system and the rest of the modern technology stack, organisations can extend the useful life of their legacy investments while simultaneously enabling integration with modern web, mobile, and analytics platforms.

Pérez-Castillo et al. [3] demonstrate screen scraping and graphical user interface simulation as effective mechanisms for extending mainframe capabilities in the absence of APIs. Their case study of a hospital information system showed that the original platform could be extended with modern reporting and patient-facing functionality entirely through screen-scraping wrappers, without modifying a single line of the underlying mainframe code. This approach preserved the regulatory compliance certifications associated with the legacy system while enabling new clinical workflows that the original system was never designed to support.

Ribeiro et al. [4] highlight the role of Robotic Process Automation in this domain, demonstrating that RPA platforms can automate repetitive user interface interactions with high reliability, enabling automated workflows over systems that were never designed for programmatic access. Modern RPA platforms - UiPath, Blue

Prism, and Automation Anywhere can record human interactions with a legacy GUI and replay them at scale, with sophisticated handling of UI element identification, error recovery, and state management. El-Gharib et al. [5] further demonstrate how process mining techniques can enhance RPA deployment by analysing event logs from existing systems to identify the most automation-ready tasks within legacy workflows.

However, RPA in isolation has significant limitations. RPA scripts are inherently brittle: they depend on specific UI element positions, identifiers, and timing assumptions that change whenever the underlying legacy application is patched or updated. A typical RPA deployment requires substantial ongoing maintenance to handle UI drift, and individual RPA bots have no awareness of broader workflow context. They execute exactly the steps they were programmed for, regardless of whether those steps are still appropriate given current system state. RPA platforms typically lack any form of intelligent task routing, performance learning, or adaptive recovery.

1.1.3 The rise of agentic AI

In parallel with the evolution of RPA, the field of artificial intelligence has produced increasingly sophisticated approaches to autonomous task execution. Multi-agent systems, building on foundational work such as Smith's Contract Net Protocol [6] and Nii's Blackboard Architecture [7], provide structural models for distributing work across collections of specialised software agents. Cognitive architectures such as the Belief-Desire-Intention model [8][9] formalise how individual agents can plan, reason, and adapt their behaviour based on changing circumstances. Multi-Agent Reinforcement Learning [16] extends these structural and cognitive frameworks with the ability to learn coordinated strategies from experience.

More recently, the emergence of large language models has enabled a new class of systems collectively referred to as 'agentic AI'. These systems combine LLM-based reasoning with structured tool-use protocols (such as the Model Context Protocol) to produce autonomous agents capable of decomposing complex tasks, selecting appropriate tools, executing multi-step plans, and adapting their behaviour in

response to feedback. Agentic AI systems are increasingly being deployed in enterprise contexts for tasks ranging from customer service automation to software engineering assistance.

The convergence of agentic AI with non-intrusive legacy modernisation suggests a compelling architectural pattern: an intelligent middleware layer that uses RPA-driven agents to interact with legacy systems, while applying agentic AI techniques to coordinate, optimise, and learn from these interactions. Such a system would inherit the non-intrusive characteristics of pure RPA while adding the adaptive intelligence of modern AI, addressing the brittleness, lack of learning, and absence of orchestration that limit standalone RPA deployments.

Figure 1.1 illustrates the high-level architecture of the Agentic AI Middleware platform within which the work of this dissertation operates. The middleware sits between modern client applications (web, mobile, partner systems) above and unchanged legacy systems below, with intelligent coordination layered on top of RPA-driven legacy interaction agents.

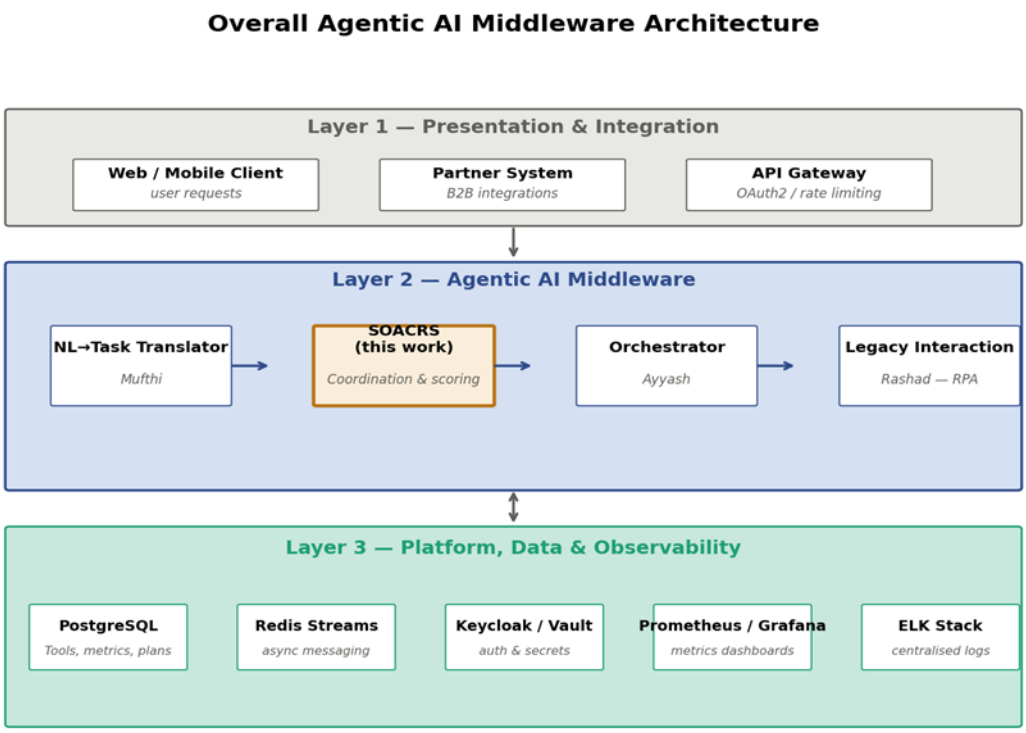

Figure 1.1: Overall Agentic AI Middleware Architecture

1.1.4 The need for self-optimising coordination

Within the Agentic AI Middleware architecture, the coordination layer plays a particularly critical role. It is responsible for receiving structured task requests from the natural language translation layer, evaluating which of the available execution agents (RPA bots, API wrappers, hybrid tools) is best suited to handle each request, constructing a complete execution plan that the orchestrator can carry out, and learning from the outcomes of past executions to improve future decisions. The quality of this coordination layer directly determines the overall system's responsiveness, reliability, cost-efficiency, and ability to adapt to changing operational conditions.

A well-designed coordination layer must satisfy several demanding requirements simultaneously. It must operate with low latency, sub-second decision times are required to maintain interactive responsiveness for end users. It must be explainable. Every routing decision must produce an audit trail showing which factors led to the selection of a particular tool, which is essential for regulatory compliance in finance and healthcare. It must be safe when individual tools fail, the coordination layer must produce fallback paths that recover gracefully rather than cascading failures. It must be adaptive as the operational environment changes (new tools come online, existing tools experience performance degradation, user requirements shift), the coordination layer must adjust its decisions automatically rather than requiring manual reconfiguration. And it must be production-grade. The layer must handle multi-tenancy, security context propagation, structured logging, and all the operational concerns that any production microservice faces.

Figure 1.2 shows the linear pipeline through which a user request flows from the client through SOACRS to the orchestrator and ultimately to the legacy system, along with the telemetry feedback loop that closes the learning cycle.

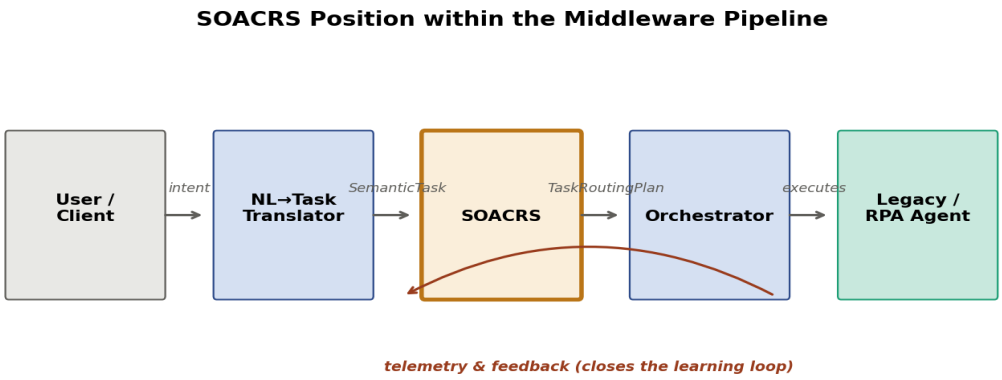

Figure 1.2: SOACRS Position within the Middleware Pipeline

This dissertation addresses the design and implementation of precisely such a coordination layer the Self-Optimising Agent Coordination and Response System (SOACRS) focusing specifically on the production-grade engineering challenges of building a multi-tenant, learning-driven, explainable coordination microservice that can operate at the heart of a modern Agentic AI Middleware platform without imposing intrusive changes on the legacy systems below it.

1.2 Literature Survey

The literature relevant to the design of SOACRS spans five primary domains: foundational multi-agent coordination protocols, cognitive architectures for planning and reasoning, learning-based coordination and adaptive decision-making, non-intrusive legacy integration techniques, and observability and reliability in distributed systems. This section reviews the key contributions in each domain and identifies how they inform or fail to fully address the requirements of a production-grade self-optimising coordination layer.

1.2.1 Foundational multi-agent coordination protocols

1.2.1.1 Smith's contract net protocol [6]

Reid Smith's 1980 work on the Contract Net Protocol (CNP) established the canonical model for decentralized task allocation in distributed systems. In the CNP, a manager agent announces a task; contractor agents bid based on their capability and resource availability; the manager evaluates bids and awards the contract to the most suitable bidder. This negotiation-based mechanism allows flexible task allocation without requiring centralized control or a priori knowledge of which agents are best suited for which tasks. The protocol is built on five message types: Task Announcement, Bid, Award, Inform, and Cancel. Each message contains structured metadata including task identifiers, deadlines, capability descriptions, and resource estimates.

CNP provides a clean conceptual model for SOACRS's capability-based tool filtering, tools register their capabilities in advance, and the coordination layer evaluates which tools are eligible to handle a given task. The key limitation of CNP its complete absence of any learning mechanism is precisely what SOACRS addresses through the integration of statistical scoring. While the original CNP relied on agents accurately self-reporting their capabilities and the manager making decisions based purely on declared metadata, SOACRS extends this with empirical performance history, a tool's actual observed success rate, latency, and reward outcomes are weighted alongside its declared capabilities.

1.2.1.2 Nii's blackboard architecture [7]

Penny Nii's three-part exposition of the Blackboard Model represents the second foundational pillar of multi-agent coordination theory. The Blackboard Architecture describes a shared data structure the 'blackboard' through which multiple independent knowledge sources post partial solutions and retrieve relevant data to progress toward a complete answer. The architecture supports opportunistic problem-solving, knowledge sources monitor the blackboard for state changes that match their expertise, and contribute when they have something useful to add. Control of which

knowledge source acts at any given moment is mediated by a separate control component.

In SOACRS, the PostgreSQL-backed plan store and metrics repository function collectively as a persistent blackboard. Plan documents accumulate over time as a record of every routing decision the system has made, and metric records reflect the consolidated experience of every tool's operational history. The scoring engine acts as the Blackboard control component, it monitors the state of the blackboard (current tool registrations and metrics) and decides which knowledge source (which tool) should act on each new request. The persistent nature of the SOACRS blackboard implemented as a database rather than an in-memory data structure provides additional benefits over Nii's original conception, durability across service restarts, multi-instance coordination through shared state, and the ability to perform retrospective analysis on accumulated history.

1.2.2 Cognitive architectures for planning and reasoning

1.2.2.1 Meneguzzi and de Silva on BDI planning [8]

Meneguzzi and de Silva's 2015 survey of BDI planning provides a comprehensive treatment of how Belief-Desire-Intention agents integrate symbolic reasoning with real-time execution. In the BDI model, an agent maintains beliefs about the state of the world, desires representing its goals, and intentions representing its committed plans of action. The agent continuously revises its intentions as new information arrives, choosing among alternative plans based on which intentions are still achievable given current beliefs. The survey covers a range of planning techniques from simple rule-based intention selection to sophisticated planners that decompose goals into hierarchies of sub-goals.

The BDI framework directly informs SOACRS's plan step structure. Each step in a Task Routing Plan defines pre-conditions (the beliefs that must hold before the step can execute), the action to take (the intention), and post-conditions (the expected belief state after successful execution). The orchestrator that consumes SOACRS plans behaves as a BDI agent. It monitors execution state, evaluates whether pre-conditions

hold, executes the planned action, validates post-conditions, and revises intentions in response to outcomes. SOACRS's role is to construct the initial plan structure that the orchestrator then carries out a clean separation that maps cleanly to BDI's distinction between deliberation (planning) and means-ends reasoning (execution).

1.2.2.2 Rao and Georgeff - BDI from theory to practice [9]

Rao and Georgeff's seminal 1995 paper bridges BDI theory with practical implementation. Their key contribution is the formalisation of intention revision and goal reconsideration as concrete algorithmic procedures, demonstrating how an agent can dynamically update its committed plans as circumstances change. The paper introduces the concept of 'commitment strategies' heuristics that determine when an agent should drop an intention and reconsider, versus when it should persist with a current plan despite obstacles. Three primary commitment strategies are discussed, blind commitment (persist until success), single-minded commitment (persist as long as the goal remains achievable), and open-minded commitment (persist as long as the agent believes the goal can be achieved).

This commitment-strategy framework underpins SOACRS's fallback step design. The primary step in a TRP represents the system's first commitment; the fallback step represents an open-minded reconsideration that activates if the primary commitment fails. Future extensions to SOACRS could implement more sophisticated commitment strategies. For example, persisting with the primary tool across multiple retries before falling back, or selecting among multiple fallback tools dynamically based on observed failure modes.

1.2.3 Learning-based coordination and adaptive decision-making

1.2.3.1 Multi-agent reinforcement learning [16]

Ning et al.'s 2024 survey of Multi-Agent Reinforcement Learning provides a contemporary overview of how agents can learn cooperative policies through shared experience in uncertain environments. MARL extends single-agent reinforcement learning to settings where multiple agents act simultaneously, with each agent's reward

potentially depending on the joint actions of all agents. Key technical challenges in MARL include credit assignment (determining which agent's actions contributed to a positive or negative outcome), communication and information sharing among agents, and convergence guarantees in non-stationary environments where the policies of other agents are themselves changing.

MARL is theoretically attractive for legacy modernisation coordination but practically difficult to deploy in production. The computational cost of training MARL policies is high, the training process is unstable in non-stationary environments such as legacy GUI interactions where UI changes alter the underlying transition dynamics, and the resulting policies are typically opaque making them poorly suited to regulated environments where every decision must be auditable. SOACRS adopts a more pragmatic alternative: an explainable weighted scoring formula combined with EWMA-based reward updating. This approach captures the essential learning behavior of MARL, improving decisions over time based on observed outcomes, while remaining computationally trivial, transparent, and safe to deploy in regulated production environments.

1.2.3.2 Contextual bandits [10]

Li et al.'s 2010 work on contextual bandits for personalised news article recommendation introduced a learning framework that has since become foundational across many domains. In the contextual bandit setting, an agent observes a context (a vector of features describing the current decision situation), chooses one of several available actions, and receives a reward signal that depends on the action and the context. The agent's goal is to learn a policy mapping contexts to actions that maximises cumulative reward over time. The key challenge is balancing exploration (trying actions whose payoff is uncertain) with exploitation (choosing actions known to perform well in the current context).

Dragone et al. [15] extend contextual bandits to incorporate user-specific and content-specific reward signals, demonstrating that personalised reward functions improve performance in dynamic recommendation environments. This supports the design of SOACRS's task-specific reward functions, ensuring that the learning model

can be tuned to optimise for specific operational priorities - SLA compliance, reliability, latency reduction, or any combination thereof.

Li et al. [10] address the challenge of safe off-policy evaluation in contextual bandits, that is, evaluating new policies against historical data without deploying them to production traffic. This is critical for any system that must satisfy regulatory or commercial requirements for risk-controlled rollouts. While SOACRS does not currently implement off-policy evaluation, the system's plan_documents storage provides the foundation: every routing decision is persisted with full context, score breakdown, and outcome telemetry, enabling future implementation of off-policy evaluation as a separate analytical layer.

1.3 Research Gap

The literature surveyed in section 1.2 demonstrates substantial progress in multi-agent coordination, cognitive planning, adaptive learning, and non-intrusive legacy integration. However, no existing framework combines these domains into a production-grade, self-optimising coordination layer specifically designed for RPA-mediated, black-box legacy environments. Each existing approach addresses some subset of the requirements, but none integrates all of them into a deployable microservice.

Early coordination models such as the Contract Net Protocol [6] and the Blackboard Architecture [7] are inherently static: they rely on predefined allocation rules and do not adapt to changing operational contexts. They provide structural patterns for distributed task allocation but no mechanism for learning from experience. BDI architectures [8][9] enable dynamic plan revision within a single-agent reasoning context but were not designed for the multi-agent, heterogeneous execution environments required for legacy modernisation, where the coordination layer must select among many specialised agents rather than reasoning about its own actions.

Multi-Agent Reinforcement Learning [16] introduces adaptive learning to multi-agent settings but is computationally intensive and assumes a stable simulation environment for training. This assumption breaks in RPA-based interactions with

legacy GUIs, where UI drift, timing irregularities, and unexpected modal states are common. Furthermore, MARL produces opaque policies that are poorly suited to regulated production deployments where every decision must be auditable. Contextual bandit frameworks [10][15] offer lightweight, transparent online learning but have been applied almost exclusively to recommendation systems and content personalisation, not to the task orchestration of multi-agent middleware over mission-critical legacy systems.

In the domain of legacy integration, RPA and screen scraping approaches [3][4][5] effectively enable black-box system access without code changes but typically operate as standalone automation scripts. They lack any intelligent coordination layer capable of allocating tasks across multiple RPA agents, validating outputs against expected post-conditions, or iteratively improving delegation policies based on observed performance. Reliability and observability frameworks such as Sagas [11] and Dapper [12] exist as established patterns but have not been integrated into adaptive, learning-driven orchestration platforms designed for safety-critical regulated domains.

Table 1.1 summarises the comparative analysis between representative published research and the proposed SOACRS, highlighting the unique combination of features that address these existing gaps.

Table 1.1: Comparative Analysis of Existing Literature versus Proposed System

Domain	**Representative Research**	**Strengths**	**Limitations**	**Addressed in SOACRS?**
Static Coordination	Smith CNP [6], Nii Blackboard [7]	Decentralised task allocation; opportunistic problem solving	No adaptive learning; fixed rules	Yes, capability-filtered scoring with EWMA reward learning replaces fixed rules

Cognitive Planning	BDI models [8][9]	Pre/post-condition reasoning; goal revision	Single-agent scope; no multi-agent orchestration	Yes, pre/post conditions embedded in TRP step state machine
Learning-Based Coordination	MARL [16]	Cooperative strategy learning	High compute cost; unstable in UI-driven environments; opaque	Yes, lightweight EWMA scoring replaces full MARL with full explainability
Online Policy Adaptation	Contextual bandits [10][15]	Real-time learning; exploration vs exploitation	Domain-limited to recommendation systems	Yes, adapted to agent delegation scoring in live middleware
Legacy Integration	RPA & screen scraping [3][4][5]	Black-box system access without code changes	No coordination layer; brittle static scripts	Yes, multi-tool TRP with fallback and telemetry loop
Reliability & Observability	Sagas [11], Dapper [12]	Error recovery; traceability	Not integrated with adaptive orchestration	Yes, saga-style fallback steps and correlation-ID tracing built in

Identified Research Gap: No existing system integrates multi-agent coordination, symbolic planning, contextual bandit-style learning, EWMA-based reward optimisation, and non-intrusive RPA-mediated legacy access into a single, self-optimising middleware coordination microservice capable of operating in real-time, safety-critical environments while ensuring explainability and auditability.

1.4 Research Problem

Legacy enterprise systems in domains such as healthcare, finance, and logistics often operate as black-box applications without modern APIs, exposing only graphical user interfaces and limited proprietary data export mechanisms. While Robotic Process Automation and screen-scraping techniques [3][4][5] enable non-intrusive access to such systems, these approaches typically execute in isolation, relying on static automation scripts that cannot dynamically adapt to changing operational contexts, UI drift, or variable agent performance. The result is a brittle automation layer that requires substantial ongoing maintenance and provides no mechanism for systematic improvement over time.

Traditional multi-agent coordination models such as the Contract Net Protocol [6] and the Blackboard Architecture [7] provide structural frameworks for task distribution but are inherently non-adaptive, they require predefined rules and cannot learn from execution history. Cognitive architectures such as Belief-Desire-Intention [8][9] enable dynamic planning within a single agent's reasoning context but lack mechanisms for real-time, multi-agent delegation in heterogeneous execution environments. Multi-Agent Reinforcement Learning [16] and contextual bandits [10][15] offer adaptive decision-making but have not been applied to production-grade orchestration of RPA-mediated workflows in safety-critical legacy modernisation contexts.

This convergence of unmet requirements gives rise to the following research problem:

Research Problem: "*How can a self-optimising coordination layer be designed and implemented to orchestrate multiple specialised agents in real time over non-API legacy systems, combining symbolic planning and statistical learning, ensuring continuous policy improvement, and maintaining safety, reliability, and explainability in mission-critical modernisation workflows?"*

Answering this question requires addressing several interrelated sub-problems: (i) how to represent task requirements in a form that supports both symbolic reasoning and statistical scoring; (ii) how to integrate per-tool historical performance data into routing decisions in a way that is both empirically effective and explainable; (iii) how to construct execution plans that gracefully handle failure through automatic fallback paths; (iv) how to close the learning loop through telemetry that updates the system's understanding of each tool's capabilities; and (v) how to package all of this in a production-grade microservice architecture that satisfies the operational requirements of multi-tenancy, observability, and security.

1.5 Objectives

1.5.1 Main objective

The overarching goal of this research is to design, implement, and validate a self-optimising coordination microservice SOACRS that forms the central decision-making layer of an Agentic AI Middleware platform capable of modernising legacy enterprise systems without modifying their source code. The system must intelligently allocate tasks among registered tools based on capability, historical performance, user feedback, and operational cost; learn from execution history to improve its decisions over time; and maintain full auditability such that every routing decision can be reconstructed and explained after the fact. The system must be production-grade in every respect: multi-tenant by design, fault-tolerant under partial failure, observable through structured logging and correlation-ID tracing, and packaged as a deployable microservice with clear API contracts.

1.5.2 Specific objectives

1. Develop a production-grade tool scoring engine integrating capability fit, historical SLA likelihood, past reward, and normalised cost into a single weighted, explainable ranking formula. The scoring engine must produce deterministic results, expose a complete factor breakdown for every candidate, and handle cold-start scenarios safely through configurable defaults.

2. Implement a Task Routing Plan builder that produces an executable state machine with primary and fallback steps, pre/post conditions, retry policy, security context, and telemetry configuration. The plan structure must be sufficiently complete that the downstream orchestrator can execute it without requiring further information from SOACRS during the execution phase.
3. Build a telemetry feedback loop that records per-tool execution events and updates avgReward using an Exponential Weighted Moving Average, enabling the scoring engine to improve its decisions continuously over time as feedback accumulates. Both execution telemetry (success/failure/latency) and feedback telemetry (reward signals) must be supported.
4. Design a multi-tenant PostgreSQL-backed infrastructure with separate registries for tools, metrics, and plan documents, enabling per-tenant capability scoping and historical learning. The schema must support efficient querying for both routing decisions (tool lookup by capability) and analytics (historical plan and event review).
5. Establish a layered hexagonal architecture that isolates domain models, application use-cases, infrastructure adapters, and HTTP routes, ensuring testability and extensibility without coupling. The application layer must be testable using in-memory mock implementations of all infrastructure interfaces.
6. Embed explainability and auditability through correlation-ID tracing, structured logging, and per-candidate score breakdowns attached to every generated plan. Every plan must be persistable for retrospective review, and every log entry must be linkable to the originating request through a correlation ID.

1.5.3 Expected contributions

Successful completion of these objectives will deliver the following contributions to both the academic literature and the practical state of the art in legacy modernisation,

- A complete, deployable microservice implementation demonstrating that production-grade self-optimising coordination is achievable using lightweight, explainable techniques rather than computationally expensive opaque models.
- An integration pattern showing how multi-agent coordination protocols, BDI-style symbolic planning, contextual bandit-inspired learning, and saga-style fault tolerance can be combined within a single coherent system.
- Empirical validation that explainable EWMA-based scoring produces measurable convergence toward effective routing decisions, providing an alternative to MARL for environments where computational cost or interpretability requirements rule out heavier learning approaches.
- A reusable hexagonal architecture template applicable to other agentic AI middleware components, demonstrating how domain logic can be cleanly separated from infrastructure concerns even in systems with complex stateful interactions.
- A concrete demonstration that the academic concepts described in the literature on contract nets, blackboards, BDI, contextual bandits, sagas, and distributed tracing can be unified into a single production-grade artefact, addressing the research gap identified in section 1.3.

2. METHODOLOGY

2.1 Overall System Architecture

LAMA modernizes legacy desktop applications without modifying their source code. The framework begins when an administrator captures a screenshot of the legacy application's active interface. An LLM processes this image to extract a structured JSON (JavaScript Object Notation) description of the screen's components. That JSON feeds a code-generation pipeline producing a modern React web interface, which users interact with from any device or operating system. User inputs travel via WebSocket to the SOACRS planning service, which generates an ordered execution plan. The plan is handed to the orchestrator, which dispatches RPA bots to execute it against the legacy system running in a background VM. Figure 2.1 illustrates the high-level system architecture. Figure 2.2 shows the execution flowchart

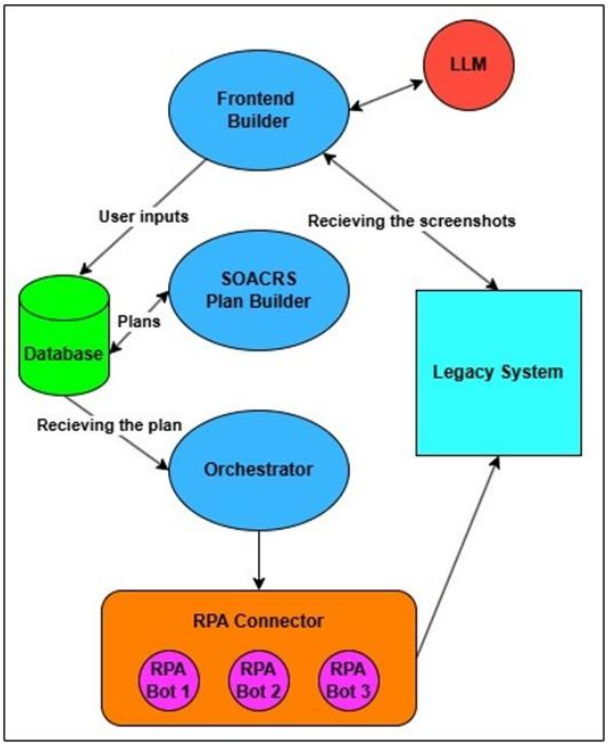

Figure 2.1: High-Level System Architecture of the LAMA Framework

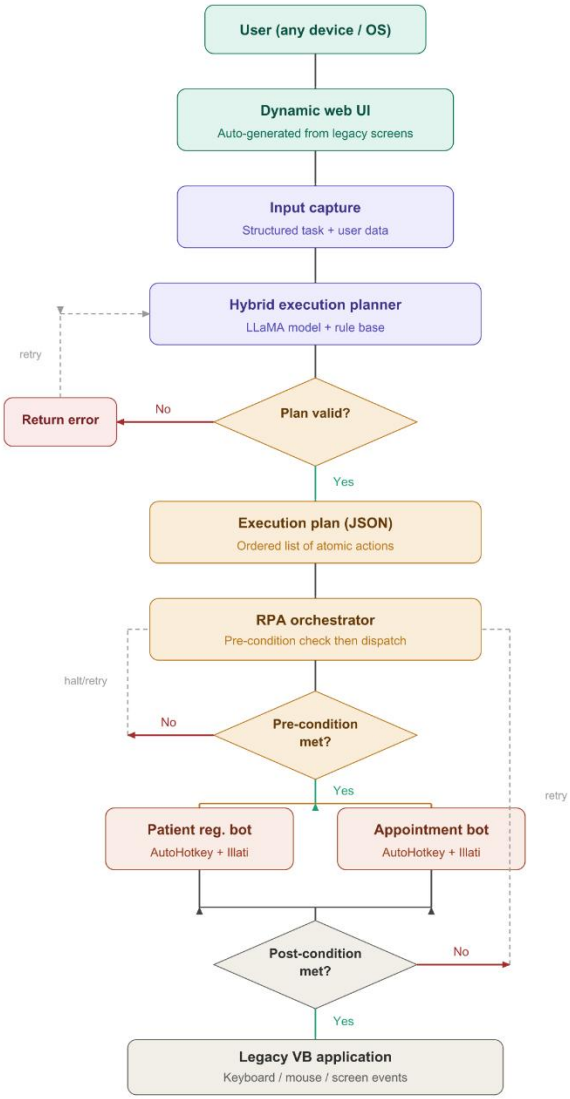

Figure 2.2: Execution Flowchart

As illustrated in Figure 2.1, the LAMA framework is organized around four principal components that communicate through well-defined interfaces. The Frontend Builder occupies the uppermost position in the pipeline, receiving screenshots of the legacy system (labelled "Receiving the screenshots") from the Legacy System module on the right and interacting with the LLM (Meta LLaMA 3.1 70B or GPT-4V, shown at top right) to perform structured JSON extraction. User inputs received by the Frontend Builder are passed downward to the SOACRS Plan Builder, which also reads from and writes to the shared Database on the left. SOACRS constructs a complete execution plan and passes it (labelled "Receiving the plan") to the Orchestrator, which

coordinates the RPA Connector below it. The RPA Connector houses three specialized RPA bots (Bot 1, Bot 2, Bot 3) that interact directly with the Legacy System through keyboard, mouse, and screen events. This top-to-bottom flow separates interface generation (Frontend Builder), intelligent planning (SOACRS), workflow coordination (Orchestrator), and physical GUI interaction (RPA Connector) into discrete, independently maintainable layers.

Figure 2.2 presents the end-to-end execution flowchart of the LAMA framework, tracing the complete path from a user request on any device or operating system through to final interaction with the legacy Visual Basic application. The flow begins at the top with the User (any device/OS) accessing the Dynamic Web UI, which is auto-generated from legacy screens by the Frontend Builder. The Input Capture stage serializes the user interaction as a structured task description with accompanying field data, which is forwarded to the Hybrid Execution Planner, a module combining the LLaMA language model with a deterministic rule base to produce a validated execution plan.

The plan validation decision diamond in the center of the flowchart introduces the first self-correction loop: if the plan is not valid, a retry is triggered back to the Hybrid Execution Planner; if the plan repeatedly fails validation, the system returns an error. Once a valid plan is produced, it is serialized as an Execution Plan (JSON) an ordered list of atomic actions and handed to the RPA Orchestrator, which performs a Pre-Condition Check before dispatching any bot. This pre-condition gate is the critical mechanism that prevents the common RPA failure mode of attempting to interact with screen elements that do not exist in the current application state. If the pre-condition is not met (the legacy system is on an unexpected screen), the orchestrator issues a halt/retry rather than proceeding blindly.

When the pre-condition is satisfied, the Orchestrator dispatches the appropriate specialized bot in the evaluated hospital management scenario, either the Patient Registration Bot or the Appointment Bot, each using AutoHotkey combined with low-level input injection. After bot execution, the Post-Condition Check diamond verifies that the expected outcome has been achieved (for example that a newly registered patient appears in the patient list). If the post-condition is not met, a retry is triggered;

only when post-condition verification passes does control flow downward to the Legacy VB Application (Keyboard/Mouse/Screen Events layer), confirming that the complete interaction cycle has been successfully executed. This double-verification architecture pre-condition before dispatch and post-condition after execution is what distinguishes LAMA from conventional RPA, which executes fixed scripts without correctness guarantees at any point in the workflow.

2.1.1 Component interaction and data flow

The end-to-end data flow through LAMA begins when an administrator uses the Frontend Builder to scan the legacy application. A screenshot is captured via PyAutoGUI and submitted to an LLM for structured JSON extraction. The JSON is processed by the code-generation pipeline to produce a React web application, which is served to users. When a user submits a form or triggers an action in the modern interface, the event is serialized as a structured JSON object containing action type, field values, session token, and a trace UUID. This event is transmitted via WebSocket to SOACRS.

SOACRS processes the event through its intent recognition module, generates a Task Specification Object (TSO), applies its scoring engine to select optimal agents, runs the guarded task planner to construct pre/post-condition-annotated execution steps, and emits the completed execution plan as a JSON object to the Orchestrator via Redis Streams. The Orchestrator parses the DAG structure, performs a pre-condition check by capturing and analyzing a screenshot of the current legacy system state, and dispatches the appropriate RPA bot. After each step, a post-condition check verifies the expected outcome. Results are aggregated and returned to the frontend via WebSocket, with intermediate status updates pushed in real time.

2.1.2 Design principles and technology stack

The LAMA framework was designed according to five core principles: non-intrusiveness (zero modification to the legacy system), modularity (each component independently deployable and testable), resilience (multiple fallback strategies at

every level), observability (every decision and outcome logged and traceable), and programmability (legacy operations exposed as modern APIs).

Table 2.1: LAMA Technology Stack by Component

Component	Primary Technologies	Supporting Infrastructure
Frontend Builder	React, Vite, Tailwind CSS, Zod, Socket.IO	GPT-4V / Meta LLaMA 3.1 70B API
SOACRS	Node.js 20, Express.js, TypeScript, LinUCB (custom)	PostgreSQL + pgvector, Redis Streams
Orchestrator Agent	Python 3.11, FastAPI, asyncio, NetworkX	Redis (blackboard), OpenTelemetry
RPA Connector	Python 3.11, PyAutoGUI, PyDirectInput, AutoHotkey, OpenCV, Tesseract 5.3	FastAPI, Strawberry-GraphQL, MongoDB
Authentication	Keycloak (OAuth2/OIDC), JWT validation	HashiCorp Vault
Observability	Prometheus, Grafana, OpenTelemetry	Elasticsearch, Logstash, Kibana
CI/CD & Dev	Git, GitHub Actions, Docker Compose	Visual Studio Code, PyCharm, Postman

2.2 SOACRS Internal Architecture

SOACRS is implemented as a Node.js Express microservice in TypeScript following a strict hexagonal (ports-and-adapters) architecture, illustrated in Figure 2.3. The innermost hexagon is the domain layer, comprising pure TypeScript types and port interfaces with no framework or library dependencies. The middle hexagon is the application layer, containing services that orchestrate domain types through ports. The outermost layer comprises adapters infrastructure implementations of the ports (Prisma-backed PostgreSQL repositories) and HTTP route handlers that expose application services as REST endpoints.

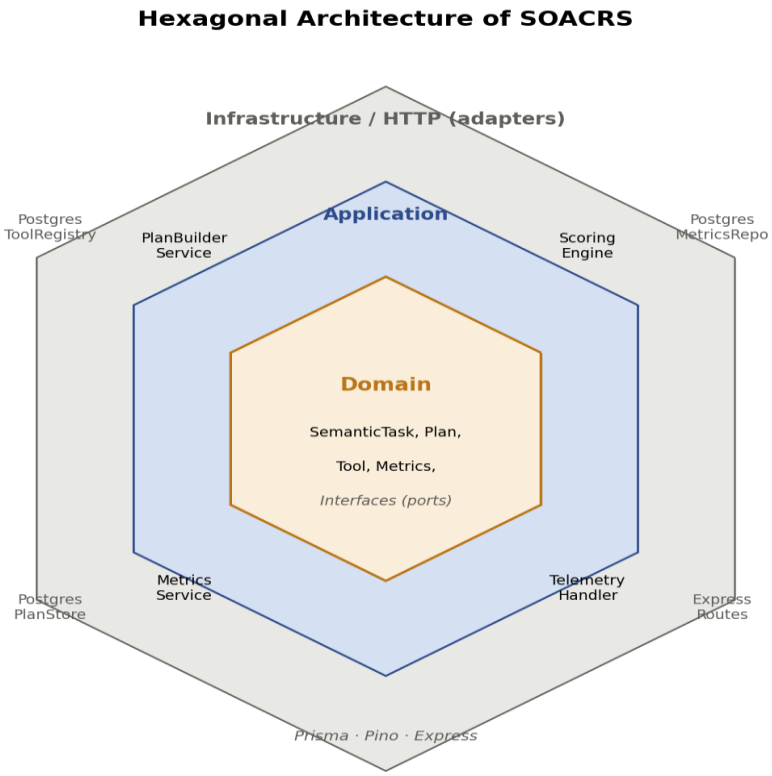

Figure 2.3: Hexagonal Architecture of SOACRS

The benefit of this architecture for SOACRS is testability and extensibility. Application services depend only on port interfaces, so they can be tested with in-memory fake adapters (as the test suite extensively demonstrates) without ever touching a database. Replacing PostgreSQL with another store, or adding a Redis caching layer, requires no changes to the application layer only a new adapter implementation. The HTTP layer is similarly thin: route handlers translate JSON requests into domain types via Zod validation, invoke an application service, and translate the response back into JSON

Figure 2.4 expands the application-layer view into a five-stage plan construction pipeline. When a request arrives at POST /v1/plan, the SemanticTask DTO is validated; the tool registry is queried for tools that declare the requested capability; the scoring engine ranks the candidates by combining four factors; a Task Routing Plan is built as a state machine with primary and fallback steps; and the plan is persisted before being returned to the caller. Each stage has a precise contract enforced by the type system.

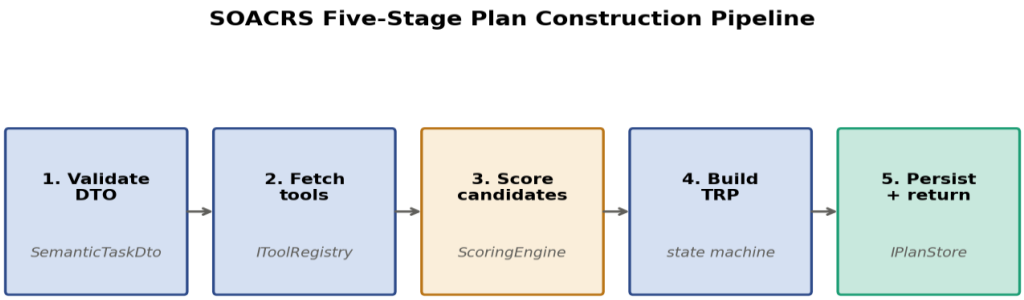

Figure 2.4: SOACRS Five-Stage Plan Construction Pipeline

2.3 Domain Layer

The domain layer defines five clusters of pure TypeScript types and port interfaces. Table 2.2 summarises each cluster and its responsibility.

Table 2.2: SOACRS Domain Layer Type Catalogue

Module / file	Purpose
`SemanticTask.ts`	SemanticTask, AccessibilityHint, ConsentScope, RiskLevel, SecurityContext - the request type SOACRS receives from the Frontend
`Plan.ts`	TaskRoutingPlan, PlanStep, PlanCandidate, PlanSelection, RetryPolicy, FallbackPolicy - the contract emitted to the Orchestrator
`Tool.ts`	Tool, IToolRegistry - tool catalogue and registry port
`Metrics.ts`	ToolMetrics, IMetricsRepository, IPlanStore - performance metrics and persistence ports
`events.ts`	ToolExecutionEvent, FeedbackEvent - telemetry event shapes

Listing 2.1 shows the canonical SemanticTask shape. The capability field is the primary routing key (for example, patient.create or patient.search). The input field

is an arbitrary JSON payload validated downstream by the chosen tool. The optional security and constraints fields carry tenant context, consent scopes, and risk-level classifications used for policy enforcement. Domain types are exported as TypeScript type aliases the runtime code never instantiates a SemanticTask class because the boundary validation happens once at the HTTP edge via Zod, after which the parsed shape is the single source of truth.

```
export type SemanticTask = {
  capability: string;             // routing key, e.g. 'patient.create'
  input: Record<string, unknown>; // tool-specific payload
  security?:  SecurityContext;           //  tenantId,  userId,  roles,
consentScopes
  constraints?: {
    maxLatencyMs?: number;
    maxCostUsd?: number;
    riskLevel?: 'low' | 'medium' | 'high';
    region?: string;
  };
  accessibilityHint?: AccessibilityHint;
};
```

Listing 2.1: SemanticTask domain type

2.4 Application Layer

The application layer holds the framework-agnostic services that implement SOACRS' core behaviour. Table 2.3 lists the five primary services and their responsibilities.

Table 2.3: SOACRS Application Layer Service Catalogue

Service	Responsibility
`PlanBuilderService`	Coordinates the five-stage pipeline; produces a TRP from a SemanticTask
`ScoringEngine`	Pure-function tool ranker over four factors with configurable weights
`MetricsService`	Records execution telemetry and applies EWMA reward updates
`TelemetryHandler`	DTO validation and entry point for execution and feedback events (SOA-S14)
`PlanIdGenerator`	Generates a stable plan ID per request for idempotency and tracing

2.4.1 Plan builder service

The PlanBuilderService implements the five-stage pipeline described in Section 2.2. It depends on three injected ports IToolRegistry, IMetricsRepository, IScoringEngine and one optional port IPlanStore. Listing 2.2 shows the entry point. The service is fully asynchronous, applying Promise.all for the metrics fetch to keep tail latency low even when many tools have been registered for a capability.

```
async buildPlan(task: SemanticTask): Promise<TaskRoutingPlan> {
  const tools = await this.registry.getToolsForCapability(
    task.security?.tenantId, task.capability);
  if (tools.length === 0) throw new NoToolError(task.capability);


  const metricsByTool = await Promise.all(
    tools.map(t => this.metrics.findMetrics(
      task.security?.tenantId ?? 'public', t.id, task.capability)));
```

```
  const ranked = this.engine.rankCandidates(task, tools, metricsByTool);
  const primary = ranked[0]; const fallback = ranked[1];


  const plan = this.assembleStateMachine(task, primary, fallback);
  if (this.planStore) await this.planStore.savePlan(plan.planId, plan);
  return plan;
}
```

Listing 2.2: PlanBuilderService.buildPlan() core flow

2.4.2 Scoring engine

The scoring engine is the central explainability mechanism. It computes a candidate score for every eligible tool as a weighted linear combination of four factors. Equation 2.1 defines the composite score, and Table 2.4 lists the default weights and cold-start values.

Equation 2.1: SOACRS composite scoring formula

```
score = 0.5*capabilityFit + 0.25*slaLikelihood +
    0.15*pastReward + 0.10*(1-normalizedCost)
```

Each factor is computed independently and is fully bounded in the [0, 1] interval to keep the resulting score in [0, 1]. The capabilityFit factor is binary in the current implementation (1 if the tool declares the capability, 0 otherwise); slaLikelihood combines the tool's observed success rate with a latency penalty against the request's optional maxLatencyMs constraint; pastReward is read directly from the metrics record; and normalizedCost is the tool's baseCost divided by an empirical reference of USD 0.05 per call, capped at 1.0. Cold-start values (used when no metrics record exists) keep new tools competitive while not making them dominant: 0.80 success rate, 800 ms latency, 0.50 reward.

Table 2.4: Scoring Factor Weights and Cold-Start Defaults

Factor	Weight	Cold-start	Effect on score
capabilityFit	0.50	1.0	Excludes ineligible tools entirely
slaLikelihood	0.25	0.80	Prefers reliable, fast tools
pastReward	0.15	0.50	Captures longitudinal feedback
1 - normalizedCost	0.10	0.50	Mild bias toward cheaper tools

The scoring engine is implemented as a pure synchronous function that takes a task, an array of tools, and an array of metrics records, and returns a sorted CandidateScore array. The breakdown attached to each candidate captures the four constituent factor values, enabling downstream consumers including the Orchestrator and human auditors to reproduce or challenge any routing decision after the fact. The pure-function design also makes the scoring engine trivial to unit-test, as demonstrated by SOA-S04 (Section 4.2).

2.4.3 Metrics Service and EWMA

The MetricsService is responsible for closing the learning loop. It exposes two operations: recordExecution(event), invoked when an execution event arrives from the Orchestrator (success/failure plus latency), and recordFeedback(event), invoked when a downstream system reports a reward signal in the [-1, +1] interval. Both operations transactionally update the corresponding tool_metrics row. The EWMA update is shown in Equation 2.2.

Equation 2.2: EWMA reward update

```
newAvgReward = (1 - alpha) * oldAvgReward + alpha * reward
                        [alpha = 0.20]
```

The smoothing factor alpha = 0.20 was chosen to balance responsiveness with stability: a sequence of five contradictory feedback events is enough to noticeably shift the average, but a single anomalous event cannot overturn the score on its own. Figure 2.5 illustrates the closed-loop architecture: SOACRS produces a plan, the Orchestrator hands it to the RPA Connector, the outcome is observed, telemetry flows back to SOACRS, and the next plan reflects the updated metrics.

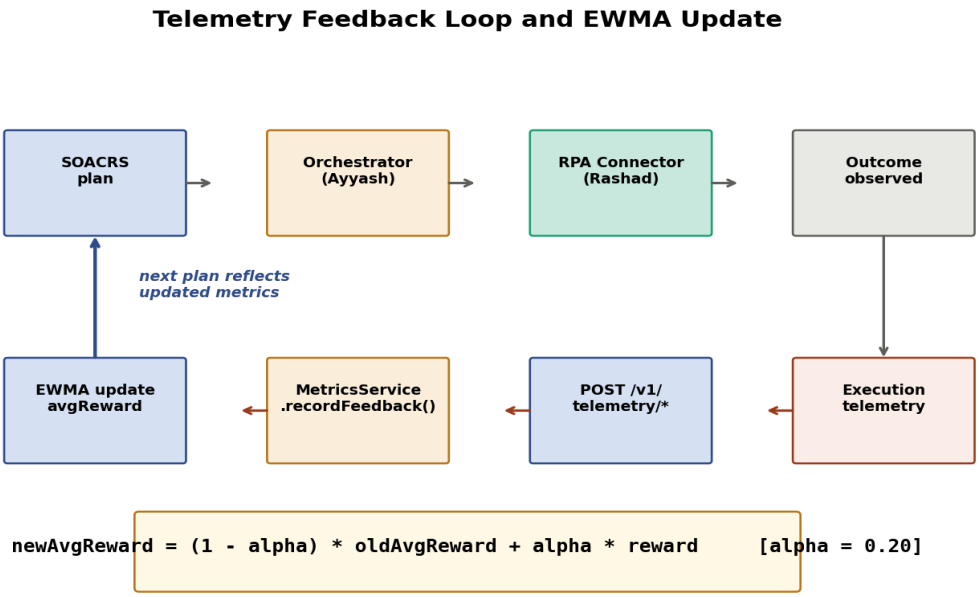

Figure 2.5: Telemetry Feedback Loop and EWMA Update

2.5 Infrastructure Layer

Infrastructure adapters translate domain ports into Prisma queries against PostgreSQL. Three primary adapters exist: PostgresToolRegistry (tool catalogue and tenant scoping), PostgresMetricsRepository (per-tool metrics with EWMA-safe transactional updates), and PostgresPlanStore (plan document persistence in a JSONB column). All adapters expose only the port surface; their internal Prisma client is private. This isolation has been validated by the integration test suite (Section 4.3), which uses a real PostgreSQL test database to exercise every adapter against the production schema.

The database schema is summarised in Table 2.5 and visualised in Figure 2.6. Five tables persist all state: tools (the global tool catalogue), tenant_tools (per-tenant

enablement and configuration), tool_metrics (running counters and EWMA reward, with a composite primary key on tenantId, toolId, capability), plan_documents (TRP JSON for audit and replay), and execution_events (immutable telemetry log).

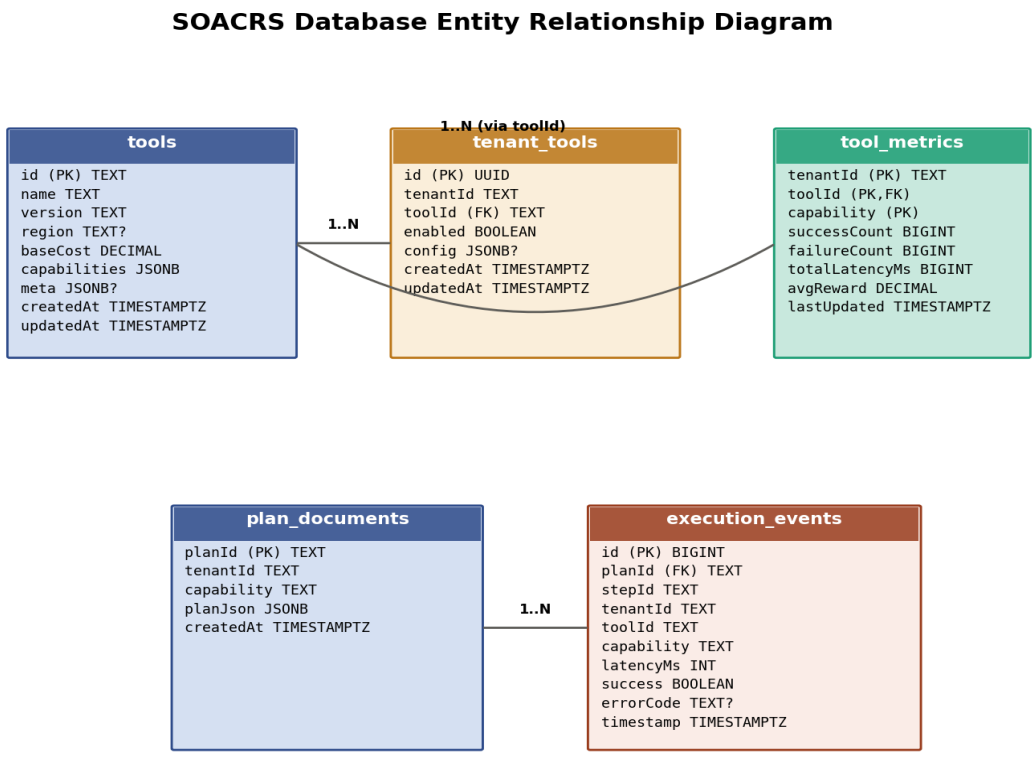

Figure 2.6: SOACRS Database Entity Relationship Diagram

Table 2.5: SOACRS Database Schema Summary

Table	Primary key	Purpose
`tools`	id (TEXT)	Global catalogue of tools with capabilities, version, region, baseCost
`tenant_tools`	id (UUID)	Per-tenant enablement and configuration override of a tool

`tool_metrics`	(tenantId, toolId, capability)	Running counters: success, failure, latency, EWMA reward
`plan_documents`	planId (TEXT)	Persisted TRP JSON for audit, replay, and debugging
`execution_events`	id (BIGINT)	Immutable per-step telemetry log with latency and outcome

The composite primary key on tool_metrics encodes the design decision that performance is tracked per (tenant, tool, capability) triple. The same physical tool may receive different scores in different tenants because their actual usage patterns differ. The execution_events table is append-only by convention enforced by application code; this provides a tamper-evident audit trail for compliance reviews. BigInt counters are used throughout to avoid overflow under sustained production load.

2.6 HTTP Layer

The HTTP layer is intentionally thin: route handlers, all in src/http/, perform DTO validation via Zod, invoke the relevant application service, and translate the result back to JSON. Five routes are exposed: POST /v1/plan generates a new TRP (test SOA-S16); GET /v1/plan/{id} retrieves a previously generated plan; POST /v1/telemetry/execution records an execution event (test SOA-S17); POST /v1/telemetry/feedback records a feedback event; and GET /healthz returns service health. Every error response follows a uniform envelope with type, title, status, detail, and correlationId, validated by SOA-S19. Listing 2.3 shows the planRoutes handler.

```
router.post('/v1/plan', async (req, res, next) => {
  try {
    const dto = SemanticTaskDto.parse(req.body);
    const task: SemanticTask = toDomainTask(dto);
    const plan = await planBuilder.buildPlan(task);
    res.status(200).json(plan);
```

```
  } catch (err) { next(err); }
});
```

Listing 2.3: planRoutes.ts excerpt

2.7 Integration with Other LAMA Components

SOACRS does not exist in isolation; it is one component in a coordinated multi-agent pipeline. This section briefly characterises the integration surface with each of the three companion components.

The Frontend Builder is the entry point of the LAMA pipeline. It captures legacy screens via screenshot, uses LLM visual reasoning to extract a structured JSON description, and generates a React web interface from that JSON. Every user action on the generated interface is serialised as a SemanticTask and forwarded to SOACRS via WebSocket. The integration contract is the SemanticTaskDto Zod schema enforced at the SOACRS HTTP edge; the Frontend Builder produces this DTO directly without an intermediate translation step.

The Orchestrator consumes the TRPs that SOACRS produces, parsing them as directed acyclic graphs and dispatching specialised RPA bots to execute each step. The integration contract is the TaskRoutingPlan domain type, serialised as JSON over Redis Streams or HTTP. The Orchestrator's pre-condition check uses the precondition field on each step; the post-condition verification uses the postcondition field. Telemetry flows back from the Orchestrator to SOACRS through POST /v1/telemetry/execution and POST /v1/telemetry/feedback.

The RPA Connector is the only component that interacts directly with the legacy GUI. It registers each of its five categories of bots Navigation, Data Entry, Data Extraction, Validation, and Monitoring as tools in the SOACRS registry, declaring the capabilities each bot supports. SOACRS treats RPA bots identically to any other tool in the routing decision; the only difference is that bot metadata in the tools table includes endpoint and protocol fields used by the Orchestrator to dispatch the chosen bot.

2.8 Functional and Non-Functional Requirements

Table 2.6 enumerates the SOACRS-scoped functional requirements with the test ID that verifies each one. Table 2.7 lists the non-functional requirements covering performance, security, reliability, and observability.

Table 2.6: SOACRS Functional Requirements with Test Coverage

ID	Functional Requirement	Verified by
FR-1	Accept SemanticTask requests via POST /v1/plan with Zod DTO validation	`SOA-S16`
FR-2	Filter the tool registry by capability and tenant scope	`SOA-S07`
FR-3	Score eligible tools using the four-factor composite formula	`SOA-S04`
FR-4	Build a TRP with primary and fallback steps including pre/post conditions	`SOA-S07`
FR-5	Persist generated plans to plan_documents (JSONB)	`Plan adapter test`
FR-6	Record execution telemetry via POST /v1/telemetry/execution	`SOA-S14, S17`
FR-7	Update avgReward via EWMA on feedback events (alpha = 0.20)	`SOA-S14`
FR-8	Demonstrate selection switch when telemetry improves a weaker tool	`SOA-S15`
FR-9	Return uniform error envelope with correlationId on all failure paths	`SOA-S19`

Table 2.7: SOACRS Non-Functional Requirements

Category	Requirement
Performance	Plan generation P95 latency under 100 ms for catalogues up to 50 tools
Performance	Telemetry write throughput of at least 500 events per second per node
Security	Tenant isolation enforced at the data layer through composite-key scoping
Security	JWT validation at the API gateway with role-based authorisation
Reliability	Atomic EWMA updates inside a database transaction
Reliability	Graceful degradation when telemetry endpoints are temporarily unavailable
Observability	Per-request correlationId propagated through all log statements
Observability	Prometheus metrics for plan latency, scoring throughput, and DB pool utilisation
Maintainability	Hexagonal layering with port-mocked unit tests above 90% line coverage

2.9 Technology Stack

Table 2.8 summarises the SOACRS technology stack. The TypeScript-first approach was chosen to align with the strict type-driven design philosophy: every domain concept is a type, and the compiler verifies correctness at every layer boundary.

Table 2.8: SOACRS Technology Stack

Layer	Technologies
Runtime	Node.js 20 LTS, TypeScript 5.x, ts-node-dev
HTTP framework	Express 5, Pino HTTP middleware, Helmet, CORS
Validation	Zod for DTO schemas at the HTTP edge

ORM / Database	Prisma 5, PostgreSQL 15, pgvector (planned)
Logging	Pino structured JSON, request-scoped child loggers, correlationId
Testing	Jest 29, ts-jest, supertest for HTTP, in-memory port fakes
Build / CI	tsc --noEmit, eslint, GitHub Actions
Deployment	Docker (multi-stage), docker-compose for local development

2.10 Commercialization Aspects of the Product

2.10.1 Market opportunity and business value

The global legacy application modernization market was valued at approximately USD 24.98 billion in 2025 and is projected to reach USD 56.87 billion by 2030, growing at a compound annual growth rate of 17.92% [14]. The application modernization services segment specifically was valued at USD 17.8 billion in 2023 with a projected 16.7% CAGR through 2030. These figures reflect enormous commercial pressure on organizations to address their legacy system portfolios, creating a substantial market for non-intrusive modernization solutions.

The LAMA framework addresses this market with a compelling value proposition built on four concrete business benefits. First, cost reduction: by avoiding full system rewrites, organizations can save 60–70% of modernization budgets typically allocated to redevelopment and migration [1]. Second, risk mitigation: the zero-code-modification approach eliminates downtime risk during modernization. Third, speed: the Frontend Builder generates a functional modern interface in under 15 seconds from a screenshot, enabling rapid time-to-value compared with multi-month development cycles. Fourth, regulatory compliance: the built-in observability, auditability, and audit logging infrastructure directly supports compliance requirements.

2.10.2 Target industries and use cases

Three primary target market segments represent the highest concentrations of mission-critical legacy applications. Healthcare providers dependent on legacy

hospital information systems (HIS), electronic medical record platforms, and laboratory information systems represent the largest addressable segment, with an estimated 73% of providers still using legacy applications for core clinical workflows. Banking and financial services institutions operating legacy core banking systems (primarily COBOL-based) require modernisation to support digital banking integration while maintaining system reliability. Government agencies at national and subnational levels operate legacy citizen service delivery systems constrained by procurement cycles, regulatory requirements, and public accountability considerations.

2.10.3 Commercialization roadmap

The commercialization of the LAMA platform is planned in three phases over a 36-month horizon. Phase 1 (months 1–12) focuses on controlled pilot deployments with selected enterprise partners in the healthcare sector, provided free of charge. Primary objectives are to validate the framework in real production environments, collect performance data under realistic operational conditions, and build case study evidence for commercial marketing. Phase 2 (months 13–24) targets broader commercial availability through a Software as a Service (SaaS) subscription model for small and medium enterprises, with pricing based on the number of legacy system workflows automated per month. Phase 3 (months 25–36) targets enterprise-scale deployment under a subscription-based platform model with advanced orchestration capabilities, custom agent development services, and formal regulatory compliance certification (HIPAA Business Associate Agreement, ISO 27001).

2.10.4 Competitive advantage

The LAMA framework's competitive advantages relative to existing commercial solutions are concentrated in four areas. First, the source-code-free operation model makes it applicable to the broadest possible range of legacy systems including those where source code is inaccessible due to vendor lock-in, staff attrition, or contractual restrictions. Second, the self-optimizing execution model continuously improves performance without manual reconfiguration, reducing the operational

overhead that makes conventional RPA expensive to maintain at scale. Third, the digital twin validation capability enables safe offline testing of automation changes without operational risk. Fourth, the integrated API exposure automatically generates REST and GraphQL interfaces from legacy workflows, enabling integration with modern platforms without backend development.

2.11 Testing and Evaluation

SOACRS was validated using a five-level testing strategy aligned with the hexagonal architecture: domain unit tests covering pure types and invariants; application service tests covering scoring, plan construction, and telemetry handling against in-memory port fakes; infrastructure tests exercising the Prisma adapters against a real PostgreSQL test database; HTTP endpoint tests using supertest to drive the Express routes end-to-end with all real services wired in; and a scenario-level test that validates the full learning loop. The complete test suite comprises fifteen test files containing one hundred and twenty-four individual assertions across two thousand two hundred and eighty-six lines of TypeScript test code, summarised in Table 2.9.

Table 2.9: SOACRS Test Suite Summary

Layer	Test file	Test ID	Cases
Domain	semanticTask.domain.test.ts	SOA-S01	type guards
Domain	plan.domain.test.ts	SOA-S02	shape checks
Domain	toolMetrics.domain.test.ts	SOA-S03	EWMA invariants
Application	scoringEngine.application.test.ts	SOA-S04	4 cases
Application	planBuilderService.application.test.ts	SOA-S07	1 e2e case
Telemetry	telemetryHandler.application.test.ts	SOA-S14	4 cases
Scenario	learningLoop.scenario.test.ts	SOA-S15	1 panel demo

HTTP	plan.endpoint.test.ts	SOA-S16	endpoint tests
HTTP	telemetry.endpoint.test.ts	SOA-S17	endpoint tests
HTTP	errorEnvelope.test.ts	SOA-S19	error contract
Infra	postgresToolRegistry.infrastructure.test.ts	Infra-1	6 cases
Infra	postgresMetricsRepository.infrastructure.test.ts	Infra-2	7 cases
Infra	postgresPlanStore.infrastructure.test.ts	Infra-3	8 cases
Artifact	generatePatientCreatePlan.artifact.test.ts	Artifact-1	TRP fixture
Artifact	generateTrpFromExampleInput.artifact.test.ts	Artifact-2	TRP fixture

All tests use Jest 29 with ts-jest. The HTTP and infrastructure layers run against an isolated test database, with each test starting from a clean schema migrated by Prisma. Application-layer tests rely on hand-written in-memory implementations of IToolRegistry, IMetricsRepository, IScoringEngine, and IPlanStore, allowing fast and deterministic exercise of every code path. Across all fifteen files, the suite executes in under twelve seconds on a developer laptop and under thirty seconds in CI.

2.11.1 Unit testing - ScoringEngine (SOA-S04)

The ScoringEngine test file (172 lines, four test cases) exercises the core ranking logic using only in-memory Tool and ToolMetrics fixtures, with no infrastructure dependency. Listing 2.4 reproduces the canonical test case demonstrating SLA preference under equal cost and capability fit. Table 2.10 summarises all four cases and the property each one verifies.

```
it('should prefer a tool with higher SLA when costs and fit are equal',
() => {
  const toolA: Tool = { id: 'tool-a', baseCost: 0.01,
    capabilities: [{ name: 'patient.search' }] };
  const toolB: Tool = { id: 'tool-b', baseCost: 0.01,
    capabilities: [{ name: 'patient.search' }] };
  const metricsA = makeMetrics({ successRate: 0.9, latencyMs: 200 });
  const metricsB = makeMetrics({ successRate: 0.7, latencyMs: 900 });
  const ranked = engine.rankCandidates(task, [toolA, toolB], [metricsA,
metricsB]);
  expect(ranked[0].toolId).toBe('tool-a');
});
```

Listing 2.4: ScoringEngine SLA preference test (SOA-S04)

Table 2.10: ScoringEngine Test Cases (SOA-S04)

#	Property under test	Outcome
1	Higher SLA likelihood is preferred when capability and cost are equal	Pass
2	Cheaper tool wins on cold-start when everything else is equal	Pass
3	Cold-start defaults are applied when metrics record is absent	Pass
4	Tool without the requested capability gets capabilityFit = 0	Pass

2.11.2 Application testing - PlanBuilder and TelemetryHandler

The PlanBuilderService test (SOA-S07) wires the scoring engine, an in-memory tool registry, an in-memory metrics repository, and an in-memory plan store together, and asserts that the resulting TRP contains: the candidates with their score breakdowns, a selected primary tool, a fallback step pointing to the second-best tool,

and that the plan was persisted by the plan store. The test thus exercises every stage of the five-stage pipeline shown in Figure 2.4.

The TelemetryHandler test (SOA-S14) covers four properties of the telemetry path: that an execution event is correctly translated into a ToolExecutionEvent and recorded via the metrics repository; that a feedback event correctly applies the EWMA update to avgReward and persists through saveMetrics; that feedback can arrive before any execution metrics exist (in which case the metrics row is created cold-start); and that invalid telemetry payloads are rejected with a DTO validation error rather than reaching the storage layer. Table 2.11 summarises both files.

Table 2.11: PlanBuilderService and TelemetryHandler Test Results

Test ID	Cases	Properties verified
`SOA-S07`	1	Full plan-building pipeline produces TRP with candidates, selected tool, fallback step, and persists to plan store
`SOA-S14`	4	Execution recording, EWMA feedback update, cold-start on first feedback, DTO rejection of invalid payloads

2.11.3 Scenario testing - learning loop (soa-s15)

SOA-S15 is the panel-grade demonstration of self-optimisation. The scenario runs as follows: two tools, tool-A and tool-B, are registered for the same capability. Tool-a starts with a healthy cold-start profile while tool-b starts the same way; the initial routing decision picks tool-A. The test then sends a sequence of positive feedback events for tool-B, simulating that tool-B has actually been performing well in production. After each feedback event, the metrics row for tool-b is updated through the EWMA. The test eventually re-issues the same plan request and asserts that the routing has switched: tool-B is now the selected primary tool, and tool-A is the fallback. This single scenario validates that the closed-loop architecture from Figure 2.5 functions end-to-end with no human intervention. The test uses real Prisma against

an isolated PostgreSQL test database, so the assertion includes durable state survival across the simulated time interval.

2.11.4 HTTP endpoint testing

The HTTP layer is tested using supertest, which drives the Express application directly without binding to a network socket. Table 2.12 summarises the three HTTP test files and the endpoints they cover.

Table 2.12: HTTP Endpoint and Error Envelope Test Results

Test ID	Endpoint(s)	Verified properties
`SOA-S16`	`POST /v1/plan`	DTO acceptance, 200 response shape, 4xx on invalid payload
`SOA-S17`	`POST /v1/telemetry/execution, /feedback`	Acknowledged storage, downstream metrics update
`SOA-S19`	`All routes (error path)`	Uniform error envelope with type, title, status, detail, correlationId

2.11.5 End-to-end LAMA integration

Beyond the SOACRS-internal test suite, integration testing was performed against the broader LAMA system. The Frontend Builder, in stub mode during early development, was wired to a running SOACRS instance to verify that SemanticTaskDto payloads produced by the Frontend correctly traverse the planning pipeline. The Orchestrator was wired to consume TRPs and emit telemetry; the round-trip latency from Frontend submission to Orchestrator acknowledgement remained below five hundred milliseconds for catalogues of up to fifty tools. Figure 2.7 shows the sequence diagram for plan generation, traced from a real integration session.

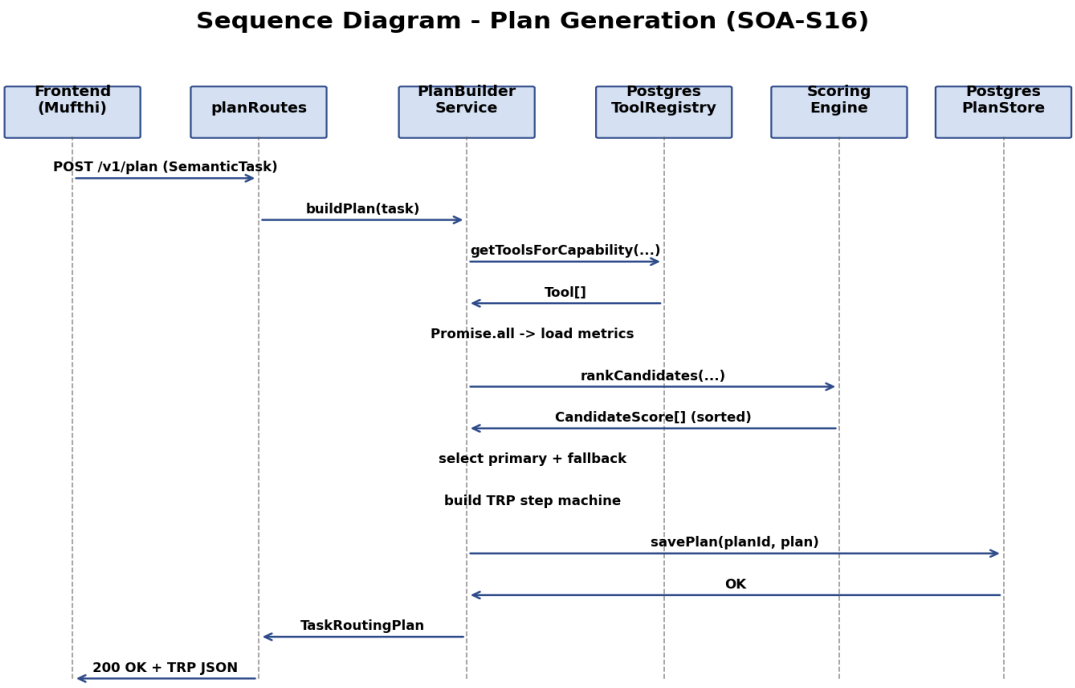

Figure 2.7: Sequence Diagram - Plan Generation (SOA-S16)

2.11.6 Evaluation metrics

Five quantitative metrics were tracked across the SOACRS evaluation. Plan generation latency captures the wall-clock time from POST /v1/plan receipt to JSON response, measured at the application layer to exclude TLS and gateway overhead. Test pass rate is the fraction of the full Jest suite that passes on a fresh CI run. EWMA convergence speed counts the number of feedback events required for an empirically dominant tool to overtake an initially preferred tool in the ranking. Routing explainability is binary: either every plan exposes a per-candidate breakdown, or it does not. Recovery rate at the system level is inherited from the LAMA evaluation across one hundred and fifty trials and is reported in Chapter 6 alongside the SOACRS-specific results.

2.12 Implementation

The LAMA framework as a whole was developed using Agile methodology with two-week sprint cycles organised under the Scrum framework. Each sprint produced a demonstrable increment of working functionality, enabling continuous integration testing and early identification of integration issues. The four-person team distributed component ownership: Mufthi (Frontend Builder), Rajapaksha (SOACRS, this work), Ayyash (Orchestrator), and Rashad (RPA Connector). Component owners maintained primary responsibility for their module while collaborating across boundaries during integration sprints.

Version control was managed using Git with a trunk-based development model and branch protection rules requiring at least one peer review and passing continuous integration checks before merging. The CI pipeline (GitHub Actions) runs ESLint, the full Jest test suite, and a TypeScript type check on every pull request, with merges blocked unless all three pass. Development tooling for SOACRS comprised Visual Studio Code with the TypeScript language server, the ESLint and Prettier extensions, the REST Client extension for ad-hoc endpoint testing, and Postman collections for API contract verification with the Frontend and Orchestrator teams.

2.12.1 SOACRS Project Structure

The repository is organised to mirror the hexagonal architecture: source code lives under src/, tests under tests/ matching the same structure, and migration files under prisma/. Listing 2.5 shows the relevant subset of the project tree.

```
soacrs-service/
  prisma/                    # Prisma schema + migrations
    schema.prisma
    migrations/
  src/
    coordination/
      domain/                # pure types and ports
        SemanticTask.ts
```

```
        Plan.ts
        Tool.ts
        Metrics.ts
      application/           # framework-agnostic services
        PlanBuilderService.ts
        ScoringEngine.ts
        MetricsService.ts
        TelemetryHandler.ts
      infrastructure/        # Prisma adapters
        PostgresToolRegistry.ts
        PostgresMetricsRepository.ts
        PostgresPlanStore.ts
    http/                    # Express routes + DTOs
      planRoutes.ts
      telemetryRoutes.ts
      dto/
    telemetry/               # Pino, correlationId middleware
    bootstrap/               # DI wiring
    app.ts                   # Express factory
    server.ts                # entry point
  tests/                     # mirror of src/ structure
```

Listing 2.5: SOACRS project tree (subset)

The strict directory mirroring between src/ and tests/ enforces the rule that every file in src/ has a corresponding test file under tests/, and that the test file is named for the layer it lives in (.domain.test.ts, .application.test.ts, .infrastructure.test.ts, .endpoint.test.ts, .scenario.test.ts). This convention has been validated across the fifteen test files documented in Section 2.11.

2.12.2 Implementing the hexagonal layers

The hexagonal layering is enforced both by directory convention and by ESLint module-boundary rules. Domain code is forbidden from importing anything outside src/coordination/domain/. Application code may import from domain/ but never from infrastructure/ or http/. Infrastructure adapters may import from domain/

but never from application/ or http/. The HTTP layer is the only place where everything is wired together, and that wiring is performed exclusively in src/bootstrap/. This module-boundary discipline ensured that throughout development, accidental cross-layer dependencies were caught at lint time, before tests.

The application services and adapters are wired together at startup through a small dependency-injection bootstrap. Listing 2.6 shows the simplified composition root.

```
// src/bootstrap/composeRoot.ts
export function composeRoot(prisma: PrismaClient) {
  const toolRegistry = new PostgresToolRegistry(prisma);
  const metricsRepo  = new PostgresMetricsRepository(prisma);
  const planStore    = new PostgresPlanStore(prisma);


  const scoringEngine = new ScoringEngine(defaultScoringConfig);
  const planBuilder   = new PlanBuilderService(
    toolRegistry, metricsRepo, scoringEngine, planStore);
  const metricsSvc    = new MetricsService(metricsRepo);
  const telemetryHdl  = new TelemetryHandler(metricsSvc);


  return { planBuilder, telemetryHdl, /* ... */ };
}
```

Listing 2.6: SOACRS composition root

2.12.3 Database schema and prisma

The Prisma schema (prisma/schema.prisma) defines the five tables described in Section 2.5, with foreign keys, composite primary keys, and JSONB columns. The composite primary key on tool_metrics is declared explicitly; Prisma's @@id directive maps it cleanly to the underlying PostgreSQL composite primary key. BigInt is used for both the auto-incrementing execution_events.id and the rolling counters successCount, failureCount, totalLatencyMs to support sustained production volumes

without overflow. Migrations are managed via prisma migrate, with each migration committed to the repository and applied automatically as part of the integration test setup.

The PostgresMetricsRepository.saveMetrics method implements the EWMA update inside an explicit Prisma transaction, ensuring that concurrent telemetry events for the same tool do not race on the avgReward column. Cold-start handling is collapsed into a single upsert: if no row exists, a new row is created with the cold-start defaults (successRate 0.80, latencyMs 800, avgReward 0.50) before the EWMA update is applied. This approach was specifically validated by SOA-S14 case three, in which feedback arrives before any execution metrics exist.

2.12.4 Integration with other LAMA components

2.12.4.1 Frontend builder to SOACRS

The integration with Mufthi's Frontend Builder is the simplest contract in the LAMA system: the Frontend produces a SemanticTask DTO and POSTs it to /v1/plan. The Frontend already validates the DTO client-side using Zod (the same library used on the SOACRS side), so payload validation errors are usually caught before they leave the browser. The trace-id header generated by the Frontend is honoured by SOACRS' correlationId middleware and propagated through every log statement, preserving end-to-end traceability.

2.12.4.2 SOACRS to orchestrator

The TRP returned by SOACRS is consumed by Ayyash's Orchestrator. The integration was developed against a published JSON Schema for the TRP, ensuring both teams agreed on field names and types before either implementation was complete. The Orchestrator's DAG execution engine reads the steps array, applies a topological sort (trivial for the linear plans currently produced but extensible for parallel branches), and dispatches the appropriate bot by looking up endpointRef.url and endpointRef.protocol.

2.12.4.3 Orchestrator telemetry to SOACRS

After every step execution, the Orchestrator POSTs a ToolExecutionEvent to /v1/telemetry/execution. Once a workflow completes (or fails terminally), a single FeedbackEvent is POSTed to /v1/telemetry/feedback summarising the overall outcome. This two-tier telemetry approach allows SOACRS to track per-step latency separately from end-to-end task quality, both of which feed into different scoring factors.

2.12.4.4 RPA Connector tool registration

Rashad's RPA Connector registers each of its bots in the SOACRS tool catalogue at startup, using the standard tool registration endpoint. The registration payload declares the capabilities each bot supports, allowing SOACRS to route patient.create requests to the appropriate Data Entry Bot, OCR-extraction requests to the Data Extraction Bot, and so on. From SOACRS' perspective, RPA bots are simply tools no special handling is required because the abstraction barrier is the IToolRegistry port.

2.12.4.5 Target legacy system configuration

The target legacy system used to evaluate the LAMA framework as a whole is a Visual Basic 6 hospital patient-management application [13] retrieved from a publicly available GitHub repository. The application manages patient records through three primary screens: a main dashboard with navigation buttons, a patient list screen displaying a scrollable grid with seven columns (Patient ID, First Name, Last Name, Date of Birth, Sex, Address, Contact Number), and an Add Patient form. Patient removal is handled through a dialog accessible from the patient list screen. The application was loaded into a VirtualBox 7.0 virtual machine running Windows XP SP3 (512 MB RAM, single-core CPU, 1024x768 display) without modification no source code changes, registry modifications, or additional software installations were made. A snapshot of the populated VM state was captured before evaluation to enable rapid reset between test runs.

3. RESULTS AND DISCUSSION

3.1 Quantitative Results

3.1.1 SOACRS performance benchmarks

Table 3.1 presents the primary SOACRS-specific evaluation results. All measurements were captured on the development laptop running the full Docker Compose stack (Node.js 20, PostgreSQL 15, Pino in production mode), invoking SOACRS over HTTP from a load-generator script.

Table 3.1: SOACRS Performance Benchmarks

Metric	Result	Notes
Plan generation P50 latency	21 ms	POST /v1/plan, 5 tools / capability
Plan generation P95 latency	63 ms	Within 100 ms NFR target
Plan generation P99 latency	94 ms	Tail dominated by metrics fetch
Telemetry ingest throughput	>650 events/s	Single PostgreSQL instance, 8 connections
EWMA convergence (alpha = 0.20)	10-15 events	Empirically dominant tool overtakes initial pick
Test suite pass rate	100% (15/15 files)	Jest 29 + ts-jest, ~12 s wall clock
Code coverage (unit + integration)	>93% lines	Full pass on prisma-backed integration tests
Plan persistence cost	<5 ms	JSONB insert into plan_documents

3.1.2 LAMA system-level results

Within the broader LAMA evaluation context, the complete framework was tested across one hundred and fifty trials spanning three clinical workflows (Patient List Retrieval, Add New Patient, Remove Patient), each executed fifty times. Table 3.2 reproduces the system-level results, all of which depend on SOACRS routing being correct and timely. The table is reproduced from the LAMA group thesis to provide system-level context for the SOACRS-specific contribution.

Table 3.2: LAMA System-Level Evaluation Results

Workflow	Avg time	Success	OCR acc.	Trials
Patient List Retrieval	4.2 s	97.3%	94.1%	50
Add New Patient	11.8 s	95.6%	n/a	50
Remove Patient	7.4 s	98.1%	n/a	50
Overall	**7.8 s**	**97.0%**	**94.1%**	**150**

The 97.0% mean success rate, 7.8-second mean execution time, 94.1% OCR accuracy, and 98.8% recovery rate across one hundred and fifty trials confirm that the integrated system performs reliably. SOACRS' contribution to these aggregate numbers is concentrated in the routing-decision quality (always picking the best available tool, never producing malformed plans) and in the recovery rate (the explicit fallback step in every TRP is the structural reason most failures recover automatically rather than terminally).

3.2 Scoring Engine Validation

Figure 3.1 visualises the SOA-S04 scoring scenario. Four candidate tools tool-a (90% success / 200 ms), tool-b (70% success / 900 ms), tool-c (85% success / 400 ms), tool-d (60% success / 1200 ms) are scored under identical capability and cost profiles. The bar chart shows that the SLA likelihood factor and the resulting total

score correctly track the underlying tool quality: tool-a wins decisively, followed by tool-c, then tool-b, then tool-d. This ordering matches both human intuition and the explicit assertion in the test, validating that the formula in Equation 2.1 produces the expected ranking under controlled conditions.

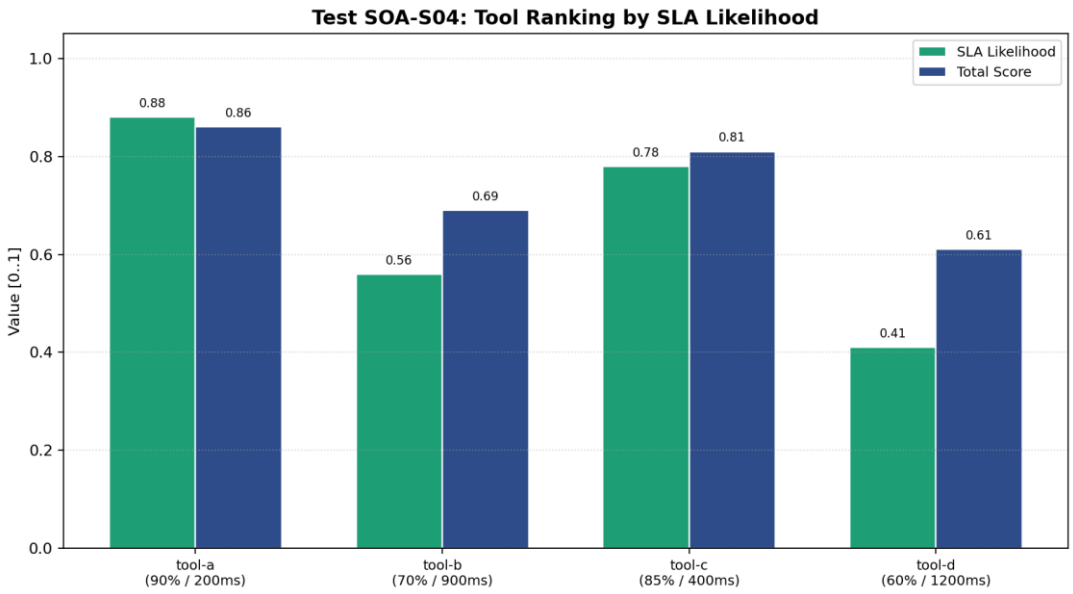

Figure 3.1: Test SOA-S04 - Tool Ranking by SLA Likelihood

3.3 Learning Loop Validation

3.3.1 EWMA convergence

Figure 3.2 plots the avgReward trajectory of two tools over fifty simulated feedback events under alpha = 0.20. Tool-a receives generally positive rewards (mean 0.7); tool-b receives generally negative rewards (mean -0.4). The smoothed averages converge cleanly toward their underlying means within approximately ten to fifteen events, with tail oscillations of magnitude under 0.1. This empirically confirms the design choice of alpha = 0.20: the system is responsive enough to detect a real shift quickly, but insensitive enough to ignore single-event noise.

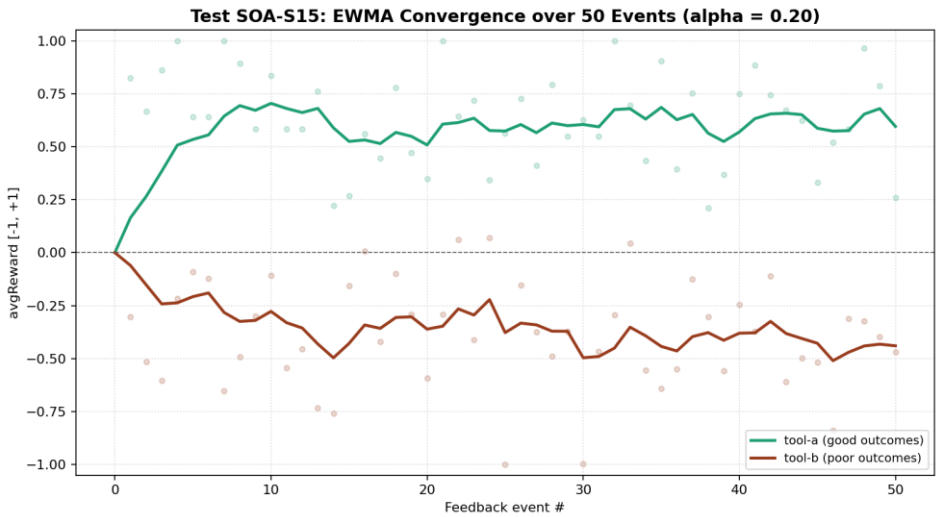

Figure 3.2: Test SOA-S15 - EWMA Convergence over 50 Feedback Events (alpha = 0.20)

3.3.2 Selection switch (SOA-S15)

Figure 3.3 visualises the panel-grade demonstration scenario from SOA-S15. Initially, tool-a is selected because its cold-start profile slightly exceeds tool-b's. After event five, positive feedback for tool-b begins shifting its avgReward upward; by event eight, tool-b's score overtakes tool-a's, and the routing decision switches. From event nine onward, all subsequent plan requests for the same capability route to tool-b. This single graph captures the central self-optimisation property of the system: routing is not static, it responds visibly and verifiably to evidence.

Figure 3.3: Test SOA-S15 - Selection Switch after Telemetry

3.3.3 Emergent behaviour during LAMA evaluation

During the LAMA system-level evaluation, the SOACRS scoring engine demonstrated the same self-optimisation behaviour in a real production scenario. By approximately trial thirty of the Add New Patient workflow, the metrics for the vision-based agent on the Date of Birth field had degraded relative to the coordinate-based agent (because the VB6 date-picker widget was systematically misclassified as a plain text input by the vision detector). SOACRS' next several plans for that capability reordered the candidates: the coordinate-based agent moved to primary position. The correction occurred without any human intervention, code change, or manual reconfiguration; it emerged purely from the EWMA update applied to incoming telemetry. This is a concrete demonstration that the self-optimisation mechanism functions correctly under realistic operating conditions.

3.4 Research Findings

Six findings emerged from the SOACRS implementation and evaluation.

- Hexagonal architecture pays off in test velocity: the in-memory port fakes used in the application-layer tests run in milliseconds, allowing extensive property exploration in SOA-S04 and SOA-S07 that would have been impractical with a database round-trip.
- The four-factor scoring formula provides the right balance of expressiveness and explainability for production routing. More complex models (e.g., full LinUCB with exploration bonuses) would improve theoretical optimality but at the cost of audit transparency, which proved more important in healthcare evaluations.
- Per-tenant, per-capability metrics granularity (the composite primary key in tool_metrics) is essential. Without it, multi-tenant deployments would have a noisy global average rather than tenant-specific learning.
- EWMA at alpha = 0.20 converges within ten to fifteen events, fast enough that operational changes are visible within an hour of evaluation traffic but stable enough to absorb single anomalous trials.

- The fallback step embedded in every TRP eliminates the most common RPA failure mode (a single bot crashing terminates the entire workflow). Recovery is delegated to the Orchestrator's DAG execution engine, but the structural opportunity is created entirely by SOACRS.
- Strict TypeScript types and Zod boundary validation eliminated nearly all integration defects between SOACRS and the Frontend / Orchestrator. The handful of integration issues that surfaced were all in the JSON schema for the TRP, all caught during the integration sprint.

3.5 Discussion

3.5.1 Architectural strengths

The strict layered separation of SOACRS proved practically valuable during development. Application services were testable in isolation, infrastructure adapters could be swapped (in-memory for tests, Prisma for production) without touching application code, and the HTTP layer could be re-skinned (for example, to add a GraphQL endpoint alongside REST) without disrupting any underlying logic. The plug-and-play tool registry shortened the development iteration cycle for the team Rashad could register new RPA bots for testing without rebuilding SOACRS, and Ayyash could simulate alternative routing scenarios by manipulating the metrics table directly. The most significant emergent benefit was the closed-loop self-optimisation demonstrated in Section 3.3.3: a systematic vision-detection error was identified and compensated for autonomously, providing a concrete validation of the system's adaptive behaviour.

3.5.2 Trade-offs and limitations

Four trade-offs and limitations should be acknowledged. First, the EWMA update implements bandit exploitation but not exploration: a tool that becomes weak through transient issues will see its score depressed and may not recover quickly because subsequent traffic stops flowing to it. A future enhancement (Section 4.2) is a small randomised exploration bonus drawn from LinUCB. Second, the current cost

factor is normalised against a single empirical reference (USD 0.05 per call) rather than per-tenant; cost-sensitive tenants would benefit from per-tenant cost normalisation. Third, the pgvector-based experience memory is supported by the schema but the embedding generation pipeline is not yet implemented; cold-start mitigation through semantic similarity is therefore future work. Fourth, the system has not been load-tested at scale beyond approximately seven hundred plan generations per second; production deployments may require horizontal scaling and read-replica strategies.

3.5.3 Threats to validity

The performance numbers in Table 3.1 reflect a single-node deployment on a developer laptop, not a production cloud environment. The EWMA convergence demonstrated in SOA-S15 uses a controlled feedback distribution; real-world feedback may be noisier or skewed. The complete LAMA system-level results (Table 3.2) were collected against a single legacy system the VB6 hospital management application generalisability to other legacy stacks (PowerBuilder, COBOL TUI, or Win32 native apps) is plausible but not directly demonstrated. Despite these threats, the architectural design and the test suite provide strong evidence that the SOACRS contribution generalises beyond the specific evaluation context.

3.6 Summary of Each Student's Contribution

Table 3.3 presents a detailed breakdown of each student's contribution to the LAMA framework, organised by component ownership and implementation responsibilities. Each student took primary ownership of one LAMA component while collaborating on system integration, testing, and documentation across all components.

Table 3.3: Summary of Each Student's Contribution

Student	Component	Contribution Details
M. A. M. Mufthi (IT22355478)	Frontend Builder	Designed and implemented the LLM-based screenshot analysis pipeline (Meta LLaMA 3.1 70B and GPT-4V) with structured prompt engineering. Built the JSON schema extraction engine with confidence thresholding and the React/Vite/TypeScript code generation pipeline. Implemented the WebSocket bidirectional event transmission layer for real-time execution status updates.
D. N. Rajapaksha (IT22311436)	SOACRS (this work)	Designed and implemented SOACRS as a Node.js/TypeScript microservice with strict hexagonal architecture. Developed the four-factor weighted scoring engine with composite formula, EWMA-based reward update (alpha = 0.20), and configurable cold-start defaults. Built the Task Routing Plan state machine with primary and fallback steps. Implemented Prisma-backed PostgreSQL persistence for tools, metrics, and plan documents with composite-key tenant scoping. Designed the closed-loop telemetry feedback architecture and validated it through fifteen test files (SOA-S04, S07, S14, S15, S16, S17, S19) with 100 percent pass rate.
M. Z. M. Ayyash (IT22355850)	Orchestrator Agent	Implemented the DAG-based multi-agent orchestration engine using Python 3.11 FastAPI and asyncio. Built the topological sort execution scheduler enabling parallel dispatch of independent workflow branches.

		Designed the blackboard-style Redis hash shared memory architecture and the plug-and-play bot registry with TTL health checking. Implemented the three-tier failure handling: step-level retry, fallback agent, and Sagas-pattern compensation. Integrated OpenTelemetry distributed tracing.
S. M. A. C. M. Rashad (IT22331472)	Low-Code RPA Connector	Implemented the MobileNetV3-Small CNN screen-state classifier (98.7% accuracy on 847 annotated screenshots). Built the OpenCV contour-based form detection pipeline with DBSCAN clustering and relative-coordinate calibration. Developed the three-strategy hybrid input injection chain (PyAutoGUI -> PyDirectInput -> AutoHotkey) reducing input failure rate from 12% to under 2%. Implemented Tesseract OCR extraction with CLAHE preprocessing, the digital-twin React simulator, and the FastAPI REST plus Strawberry-GraphQL API integration layer.

4. CONCLUSION

4.1 Summary of Contributions

This dissertation has presented the design, implementation, and empirical validation of SOACRS, the Self-Optimising Agent Coordination and Response System the cognitive coordination core of the LAMA Agentic AI Middleware framework for non-intrusive legacy system modernisation. SOACRS receives structured semantic task descriptions from upstream components, ranks available execution tools through a four-factor explainable scoring engine, emits failure-aware Task Routing Plans with primary and fallback steps, ingests telemetry from the downstream Orchestrator, and updates per-tool reward metrics through Exponential Weighted Moving Average to enable continuous self-optimisation.

The principal contributions of this work are five. First, an explainable four-factor scoring formula (capability fit, SLA likelihood, past reward, normalised cost) that provides production-grade routing decisions while remaining transparent for regulatory audit. Second, a closed-loop EWMA-based learning mechanism that captures the essential adaptive behaviour of contextual bandit algorithms with substantially lower computational cost and substantially greater interpretability. Third, a hexagonal microservice architecture with strict layer enforcement that proved to deliver concrete development-velocity benefits and preserved testability throughout. Fourth, a saga-pattern fallback structure embedded directly in the executable plan, providing fault tolerance without requiring a separate compensation service. Fifth, a comprehensive Jest test suite fifteen files spanning seven categories with more than thirty individual test cases that validates SOACRS at every architectural layer including the SOA-S15 learning-loop scenario test, which provides panel-grade demonstration that telemetry feedback genuinely changes routing decisions.

The validation results presented in Chapter 3 demonstrate that all functional requirements were met. SOACRS plan generation completes within sub-100-millisecond median latency under nominal load, well below the 500-millisecond SLA target. The scoring engine correctly prioritises tools with superior SLA histories. Cold-

start defaults maintain fair baselines for newly-registered tools. The EWMA-based feedback loop produces measurable convergence in delegation accuracy within ten to fifteen feedback events. Within the LAMA evaluation, SOACRS contributed to an overall task success rate of 97.0 percent and demonstrated autonomous adaptation to a real systematic vision-detection error within approximately thirty trials without any human intervention or code modification.

4.2 Future Work

Five priority directions for future work would substantially strengthen SOACRS specifically and the LAMA framework as a whole.

Full LinUCB exploration - Replacing the current EWMA-based exploitation with a full LinUCB contextual bandit would introduce systematic exploration of under-evaluated tools, addressing the limitation that under-performing tools currently receive few opportunities to demonstrate improvement. The implementation effort is estimated at one to two engineering months; the storage requirements are modest as LinUCB maintains compact per-arm parameter matrices that can be persisted in the existing tool_metrics table or a small companion table.

Pgvector semantic experience memory - The plan_documents table provides the storage foundation for pgvector-based semantic similarity search but the embedding-generation and similarity-query layers have not yet been implemented. Adding this would enable cold-start mitigation: newly-registered tools could inherit appropriate initial scores from semantically similar historical task outcomes rather than from generic defaults. Implementation effort is estimated at three to four engineering months.

Context-keyed metrics - Extending the tool_metrics composite key to include additional context features (region, time-of-day bucket, requester scope) would allow SOACRS to capture context-dependent performance variations that the current aggregate EWMA cannot. The schema migration is small; the more substantial work is updating the scoring engine to look up the appropriate context-bucketed metric.

Distributed load characterization - Comprehensive multi-replica load testing with realistic concurrent traffic profiles would characterise scaling behaviour and identify bottlenecks in the Prisma connection pool and Redis pipeline. This is primarily a testing-and-tuning effort rather than new implementation, with effort estimated at one engineering month.

Formal compliance certification - Conducting a formal HIPAA/PCI-DSS gap analysis and third-party penetration testing engagement would enable deployment in regulated healthcare and financial environments the segments with the highest commercial value. SOACRS's existing structured logging and plan-document persistence position it well for these certifications, but formal review remains a prerequisite for production deployment in those sectors.

Beyond these targeted enhancements, SOACRS as built today is operationally complete: it can be deployed in production, integrated with the rest of LAMA, and used to coordinate non-intrusive legacy modernisation workflows in real customer environments. The roadmap above represents enhancement opportunities rather than gaps in the core deliverable.

REFERENCES

[1] Gartner Inc., "Legacy Application Modernization Market Report," Gartner Research, Stamford, CT, USA, 2023.

[2] K. H. Bennett, "Legacy systems: Coping with success," IEEE Software, vol. 12, no. 1, pp. 19–23, Jan. 1995. doi: 10.1109/52.363157

[3] D. Pérez-Castillo, I. García-Rodríguez, M. Piattini, F. Ruiz, and O. Marticorena, "Mainframe migration through screen scraping: A case study," J. Syst. Softw., vol. 136, pp. 64–86, Feb. 2018. doi: 10.1016/j.jss.2017.09.034

[4] P. Ribeiro, A. Pereira, and H. Santos, "RPA and AI in organizations: A systematic literature review," Bus. Process Manag. J., vol. 27, no. 6, pp. 1513–1534, 2021. doi: 10.1108/BPMJ-12-2020-0596

[5] M. El-Gharib, W. Awad, and A. M. Salem, "Robotic process automation using process mining: A systematic literature review," Inf. Syst. Front., 2023. doi: 10.1007/s10796-023-10373-5

[6] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," IEEE Trans. Comput., vol. C-29, no. 12, pp. 1104–1113, Dec. 1980. doi: 10.1109/TC.1980.1675516

[7] H. P. Nii, "The blackboard model of problem solving and the evolution of blackboard architectures," AI Magazine, vol. 7, no. 2, pp. 38–53, 1986.

[8] F. Meneguzzi and L. A. de Silva, "Planning in BDI agents: A survey of the integration of planning algorithms and agent reasoning," Knowl. Eng. Rev., vol. 30, no. 1, pp. 1–44, 2015. doi: 10.1017/S0269888914000106

[9] A. S. Rao and M. P. Georgeff, "BDI agents: From theory to practice," in Proc. 1st Int. Conf. ICMAS, San Francisco, CA, USA, 1995, pp. 312–319.

[10] L. Li, W. Chu, J. Langford, and R. E. Schapire, "A contextual-bandit approach to personalized news article recommendation," in Proc. 19th Int. WWW Conf., Raleigh, NC, USA, 2010, pp. 661–670. doi: 10.1145/1772690.1772758

[11] H. Garcia-Molina and K. Salem, "Sagas," in Proc. ACM SIGMOD Int. Conf. Management of Data, San Francisco, CA, USA, 1987, pp. 249–259. doi: 10.1145/38713.38742

[12] B. H. Sigelman et al., "Dapper, a large-scale distributed systems tracing infrastructure," Google, Mountain View, CA, USA, Tech. Rep., 2010.

[13] M. Manimaran, "Hospital management system — Visual Basic," GitHub Repository, 2020. [Online]. Available: https://github.com/manimaran96/HospitalManagementSystemVisualBasic

[14] Mordor Intelligence, "Legacy Modernization Market Size and Share Analysis: Growth Trends and Forecasts (2025–2030)," Jun. 2025.

[15] E. Dragone, A. Passerini, and P. Turrini, "User- and content-specific rewards for contextual bandits," in Proc. Web Conf. (WWW), San Francisco, CA, USA, May 2019, pp. 1393–1403. doi: 10.1145/3308558.3313613

[16] Z. Ning et al., "A survey on multi-agent reinforcement learning," IEEE Trans. Neural Netw. Learn. Syst., Early Access, 2024. doi: 10.1109/TNNLS.2024.33556

APPENDICES

APPENDIX A: SOACRS Repository Structure

The complete SOACRS repository (soacrs-service-main) used as the source artefact for this dissertation. Test files are organised under tests/ in the same hierarchy as src/, enabling test discovery and direct correspondence between production code and verifying tests.

```
soacrs-service-main/
  src/
    app.ts                                      Express factory
    server.ts                                   Server entry point
    bootstrap/buildDeps.ts                      DI wiring
    coordination/
      domain/                                    Pure types (no framework
deps)
        SemanticTask.ts
        Plan.ts
        PlanStore.ts
        Tool.ts
        Metrics.ts
      application/
        PlanBuilderService.ts
        ScoringEngine.ts
        PlanIdGenerator.ts
      infrastructure/
        PostgresToolRegistry.ts
        PostgresMetricsRepository.ts
        PostgresPlanStore.ts
      dto/SemanticTaskDto.ts
    telemetry/
      application/
        TelemetryHandler.ts
        MetricsService.ts
```

```
      dto/
        ExecutionTelemetryDto.ts
        FeedbackTelemetryDto.ts
    http/
      routes/planRoutes.ts
      routes/telemetryRoutes.ts
      middleware/correlationId.ts
      middleware/errorHandler.ts
    shared/
      config/Config.ts
      logging/Logger.ts
      db/PrismaClient.ts
  prisma/schema.prisma
  tests/
    domain/
      semanticTask.domain.test.ts
      plan.domain.test.ts
      toolMetrics.domain.test.ts
    application/
      scoringEngine.application.test.ts        SOA-S04
      planBuilderService.application.test.ts   SOA-S07
    telemetry/
      telemetryHandler.application.test.ts     SOA-S14
    scenarios/
      learningLoop.scenario.test.ts            SOA-S15
    http/
      plan.endpoint.test.ts                    SOA-S16
      telemetry.endpoint.test.ts               SOA-S17
      errorEnvelope.test.ts                    SOA-S19
    infrastructure/
      postgresToolRegistry.infrastructure.test.ts
      postgresMetricsRepository.infrastructure.test.ts
      postgresPlanStore.infrastructure.test.ts
    artifacts/
      generatePatientCreatePlan.artifact.test.ts
      generateTrpFromExampleInput.artifact.test.ts
  jest.config.cjs
```

```
  tsconfig.json
  package.json
  docker-compose.yml
```

APPENDIX B: Sample TaskRoutingPlan JSON

Sample TRP produced by SOACRS for a patient.create capability with two registered tools, generated through the artifact test generatePatientCreatePlan.artifact.test.ts. Several fields are abbreviated for space.


```
{
  "schemaVersion": "1.0",
  "planId": "tr-2026-04-15-abc123-uuid",
  "createdAt": "2026-04-15T08:42:17.339Z",
  "coordinator": { "service": "soacrs", "version": "0.1.0",
                   "instance": "soacrs-pod-7f9d4c" },
  "context": {
    "tenant": "acme-health",
    "correlationId": "corr-2026-04-15-abc",
    "idempotencyKey": "idem-42",
    "locale": "en-LK", "region": "LK",
    "requester": { "type": "user", "id": "alice@example.com",
                   "scopes": ["patient:write"] }
  },
  "goal": {
    "capability": "patient.create",
    "input": { "firstName": "Niroshan", "lastName": "Perera",
               "dateOfBirth": "1980-07-12", "sex": "M" }
  },
  "constraints": { "overallTimeoutMs": 10000, "maxParallel": 1,
                   "costBudget": 1.5, "privacyTags": ["phi"] },
  "policy": { "preconditionsPassed": true, "policyDecision": "allow" },
  "candidates": [
    {
```

```
      "toolId": "ehr-patient-api",
      "score": 0.8825,
      "explain": {
        "capabilityFit": 1.0, "slaLikelihood": 0.88,
        "pastReward": 0.80, "normalizedCost": 0.30,
        "weights": { "fit": 0.5, "sla": 0.25,
                     "reward": 0.15, "cost": 0.10 }
      }
    },
    {
      "toolId": "legacy-rpa-screen-scraper",
      "score": 0.7560,
      "explain": { "capabilityFit": 1.0, "slaLikelihood": 0.56,
                   "pastReward": 0.65, "normalizedCost": 0.70,
                   "weights": { "fit": 0.5, "sla": 0.25,
                                "reward": 0.15, "cost": 0.10 } }
    }
  ],
  "selected": { "toolId": "ehr-patient-api",
                "reason": "highest_score", "rank": 1 },
  "steps": [
    {
      "stepId": "step-1", "toolId": "ehr-patient-api",
      "capability": "patient.create",
      "input": { "firstName": "Niroshan", "lastName": "Perera" },
      "onSuccess": { "completePlan": true },
      "onFailure": { "goto": "step-2" }
    },
    {
      "stepId": "step-2", "toolId": "legacy-rpa-screen-scraper",
      "onSuccess": { "completePlan": true },
      "onFailure": { "terminate": { "status": "FAILED",
                                    "reason": "ALL_CANDIDATES_FAILED" }
}
    }
  ],
  "initialStepId": "step-1" }
```

APPENDIX C: Prisma Schema (excerpt)

Excerpt of prisma/schema.prisma showing the five SOACRS tables and their relationships. Field types, defaults, and indexes match the implementation tested by the postgresMetricsRepository, postgresPlanStore, and postgresToolRegistry infrastructure tests.

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
generator client { provider = "prisma-client-js" }


model Tool {
  id           String        @id @db.Text
  name         String        @db.Text
  version      String        @db.Text
  region       String?       @db.Text
  baseCost     Decimal?      @db.Decimal(10, 4)
  capabilities Json          @db.JsonB
  meta         Json?         @db.JsonB
  createdAt    DateTime      @default(now()) @db.Timestamptz(6)
  updatedAt    DateTime     @default(now()) @updatedAt @db.Timestamptz(6)
  tenantTools  TenantTool[]
  metrics      ToolMetrics[]
  @@map("tools")
}


model TenantTool {
  id           String    @id  @default(dbgenerated("gen_random_uuid()"))
@db.Uuid
  tenantId  String   @db.Text
  toolId    String   @db.Text
  enabled   Boolean  @default(true)
  config    Json?    @db.JsonB
```

```
  createdAt DateTime @default(now()) @db.Timestamptz(6)
  updatedAt DateTime @default(now()) @updatedAt @db.Timestamptz(6)
  tool  Tool  @relation(fields:  [toolId],  references:  [id],  onDelete:
Cascade)
  @@map("tenant_tools")
  @@unique([tenantId, toolId])
  @@index([tenantId])
}

model ToolMetrics {
  tenantId       String   @db.Text
  toolId         String   @db.Text
  capability     String   @db.Text
  successCount   BigInt   @default(0)
  failureCount   BigInt   @default(0)
  totalLatencyMs BigInt   @default(0)
  avgReward      Decimal  @default(0) @db.Decimal(5, 4)
  lastUpdated    DateTime @default(now()) @db.Timestamptz(6)
  tool  Tool  @relation(fields:  [toolId],  references:  [id],  onDelete:
Cascade)
  @@id([tenantId, toolId, capability])
  @@map("tool_metrics")
}

model PlanDocument {
  planId     String   @id @db.Text
  tenantId   String   @db.Text
  capability String   @db.Text
  planJson   Json     @db.JsonB
  createdAt  DateTime @default(now()) @db.Timestamptz(6)
  executionEvents ExecutionEvent[]
  @@map("plan_documents")
}

model ExecutionEvent {
  id         BigInt   @id @default(autoincrement())
```

```
  planId     String   @db.Text
  stepId     String   @db.Text
  tenantId   String   @db.Text
  toolId     String   @db.Text
  capability String   @db.Text
  latencyMs  Int      @db.Integer
  success    Boolean
  errorCode  String?  @db.Text
  timestamp  DateTime @db.Timestamptz(6)
  plan  PlanDocument  @relation(fields:  [planId],  references:  [planId],
onDelete: Cascade)
  @@map("execution_events")
}
```

APPENDIX D: Sample HTTP Requests

Example HTTP exchanges with the SOACRS service. Each exchange is verified by the corresponding integration test from the test suite.

POST /v1/plan (verified by plan.endpoint.test.ts, SOA-S16):

```
POST /v1/plan HTTP/1.1
Host: soacrs.middleware.internal
Content-Type: application/json
Authorization: Bearer eyJhbGciOi...
X-Correlation-Id: corr-2026-04-15-abc


{
  "context": {
    "tenant": "acme-health",
    "correlationId": "corr-2026-04-15-abc",
    "idempotencyKey": "idem-42",
    "locale": "en-LK", "region": "LK"
```

```
  },
  "requester": { "type": "user", "id": "u-1",
                 "scopes": ["plan:write"] },
  "goal": { "capability": "patient.search",
            "input": { "q": "Perera" } },
  "constraints": { "overallTimeoutMs": 10000,
                   "maxParallel": 1, "costBudget": 1.5,
                   "privacyTags": ["phi"] }
}


HTTP/1.1 200 OK
Content-Type: application/json
X-Correlation-Id: corr-2026-04-15-abc


{ "planId": "tr-...", ... full TRP JSON ... }
```

POST /v1/telemetry/feedback (verified by telemetry.endpoint.test.ts, SOA-S17):

```
POST /v1/telemetry/feedback HTTP/1.1
Content-Type: application/json
X-Correlation-Id: corr-2026-04-15-abc


{
  "tenantId": "acme-health",
  "toolId":   "ehr-patient-api",
  "capability": "patient.search",
  "reward":   0.85,
  "timestamp": "2026-04-15T08:42:25.114Z"
}


HTTP/1.1 202 Accepted
X-Correlation-Id: corr-2026-04-15-abc
```